Proiect Ingineria Programării

Auto Parts Store Management App

Realizat de:

Lefter Andrei, Pitic Emanuel, Samachiș Eduard-Iulian, Sfichi Alin

Grupa 1308A

Software Requirements Specification

for

Auto Parts Store Management App

Version 1.0 approved

Prepared by Lefter Andrei, Pitic Emanuel, Samachiș Eduard Iulian, Sfichi Alin

“Gheorghe Asachi” Technical University of Iasi

30.06.2025

Table of Contents

Revision History

Name	Date	Reason For Changes	Version
Initial Release	30.05.2025	-	1.0

1. Introduction

1.1 Purpose

The purpose of this project is to develop a simple and functional windows application for an auto parts shop. The application is meant to make it easier to manage employee accounts, item stocks while separating access on levels. By separating responsibilities, the system helps reduce human error, and secure data access. With this app, we're aiming to give the shop a more organized way of running things.

1.2 Document Conventions

In this document, we followed a standard structure inspired by the IEEE SRS format. Section titles are numbered for easy navigation (e.g., 1.1, 2.3, etc.). We used simple and clear language to make the document easy to read.

1.3 Intended Audience and Reading Suggestions

This document is intended for all participants on the development of the autoparts shop management system.

- **Developers**: To understand the required features, system behavior, and how permission levels affect access to different parts of the application.
- **Testers**: To design and execute test cases based on the defined functional and non-functional requirements.
- **Users**: To get a clear understanding of what the system will do, and how it supports stock management, sales, and user access control.

The remainder of this Software Requirements Specification (SRS) is organized into several key sections, including:

- An overview of the system and its purpose
- Functional requirements specifying the system's behavior
- Non-functional requirements covering performance, security, and usability

1.4 Product Scope

The software described in this document is a management system designed for use in an auto parts shop. The main goal of the system is to improve the efficiency and accuracy of the shop's daily operations by digitizing stock control, sales processing, and user access management. The system provides a secure and role-based environment where each account can perform only the actions permitted by its assigned access level.

This software aligns with common business goals such as improving operational efficiency, reducing time spent on repetitive tasks, and maintaining accurate records of inventory.

1.5 References

N/A

2. Overall Description

2.1 Product Perspective

The Auto Parts Shop Management System is a new, self-contained desktop application designed specifically for small to medium-sized auto parts retailers, meant to support daily operations such as inventory control, sales handling and account management. It's a way of digitizing tasks that may currently be done manually or across many different tools (e.g., paper records, spreadsheets, etc.).

The system is being developed as a standalone product, with all business logic, user interfaces, and data storage implemented within a single WinForms-based C# desktop application. It will connect to a local or networked SQL database for persistence of data.

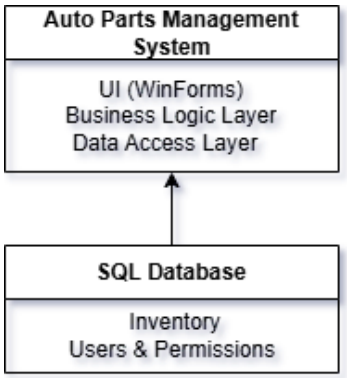


Fig. 1. System overview

2.2 Product Functions

The Auto Parts Shop Management System must support the following core functions, grouped by functional area:

- **Inventory management:**
  - Add, restock or change the price of inventory items
  - View current stock and details of items

- **Sales management:**
  - Sell inventory items
- **User management:**
  - Create and update accounts
  - Password management
  - Role-based access control (e.g., Admin, Salesperson)

2.3 User Classes and Characteristics

This section describes the different types of users who will interact with the Auto Parts Shop Management System. Users are grouped into classes based on how often they use the system, what parts of the system they use, and their level of access or experience. Some features of the system will only be available to certain user classes.

1. **Administrators**
   - **Frequency of Use**: Daily
   - **Functions Used**: User account management
   - **Technical Expertise**: High
   - **Privilege Level**: Limited to user management functions
   - **Pertinent Characteristics**: Responsible for creating, modifying, and deactivating user accounts, assigning appropriate roles, and ensuring that access permissions align with organizational policies.
   - **Importance:** High - they ensure that only authorized personnel can access specific parts of the system, which is essential for maintaining security and operational integrity.
2. **Sales Staff (Cashiers / Counter Staff)**
   - **Frequency of Use:** Daily
   - **Functions Used:** Sales processing, inventory lookup
   - **Technical Expertise:** Basic
   - **Privilege Level:** Limited to sales-related modules
   - **Pertinent Characteristics:** Interact directly with customers, handle transactions, and need quick, intuitive access to product and pricing information.
   - **Importance:** High - essential for daily business operations.
3. **Inventory Managers**
   - **Frequency of Use:** Frequently
   - **Functions Used:** Stock level monitoring, restocking, product entry and categorization, price updating.
   - **Technical Expertise:** Basic
   - **Privilege Level:** Limited to stock-related modules
   - **Pertinent Characteristics:** Ensure product availability, accuracy in inventory records, and timely restocking.
   - **Importance:** High - system effectiveness depends on accurate inventory data.

2.4 Operating Environment

The application is based on Winforms, meaning it can only run on machines running Windows operating systems.
Minimum system specifications:

- Windows 10 or later
- 8 GB of RAM
- 50 MB of free disk storage
- 1 GHz processor
- .NET 8.0 or later installed on the machine

2.5 Design and Implementation Constraints

This section outlines the main constraints that could affect how the developers design and build the Auto Parts Shop Management System. These limitations come from different sources, like hardware, software, company policies, or external requirements, and they need to be considered throughout the development process.

1. **Hardware Limitations:**
   The system is expected to run on standard desktop computers with average processing power and limited memory. This means developers need to keep performance in mind and avoid creating features that are too resource-heavy.
2. **Security Requirements**
   User authentication and access control are essential. Each user must be able to access only the parts of the system that match their role. Also, all sensitive data (like user credentials and sales records) must be stored securely, following basic encryption and security practices.

2.6 User Documentation

The software includes a built-in HelpNDoc guide, accessible directly through the user interface, which provides step-by-step instructions for using the system.
In addition to this integrated help system, a comprehensive User Manual in PDF format will be provided. The manual covers installation procedures, feature descriptions, user roles, and common workflows. Both the HelpNDoc guide and the PDF manual are designed to help users navigate the system easily and perform tasks efficiently. The documentation will follow a clear structure with concise explanations to ensure usability and consistency.

2.7 Assumptions and Dependencies

This section outlines the key assumptions and dependencies that may influence the requirements described in this Software Requirements Specification (SRS). These are not confirmed facts, but rather expected conditions that, if proven false or changed, could impact the development process or final outcome of the project.
Assumptions:

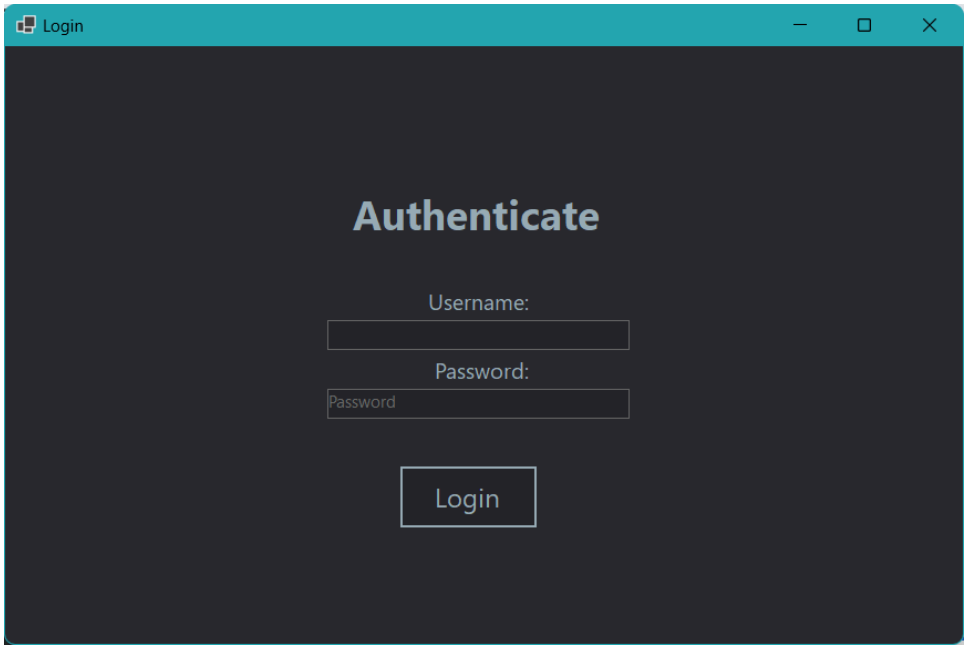
- **Operating Environment:**
  It is assumed that the software will run on Windows-based desktop systems with internet access and standard hardware capabilities (moderate RAM, processing power, and disk space). If the actual environment differs, performance or compatibility issues may arise.
- **User Access:**
  It is assumed that users will have basic computer literacy and access credentials appropriate to their roles. If users require more training or if access roles change, the system's usability and permission structure may need to be adjusted.
- **Stable Requirements:**
  It is assumed that the core functional requirements will remain stable during the main phases of development.

3. External Interface Requirements

3.1 User Interfaces

The application provides dedicated user interfaces for three categories of users: Administrator, Stock Manager and Sales Representative. Each interface is tailored to the specific responsibilities and workflows of the corresponding user role.

The application first opens a User Login Interface that requires the user to enter a valid username and password combination.

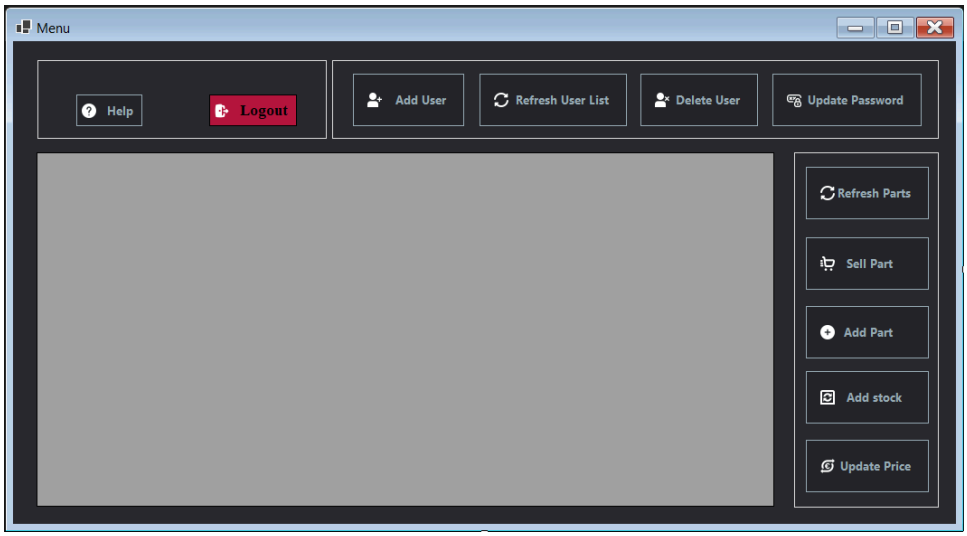


Login interface

Upon successful login, the program transitions to a Menu Interface, otherwise, an error message will be displayed. The only common element across all user interfaces is the top-left bar, which

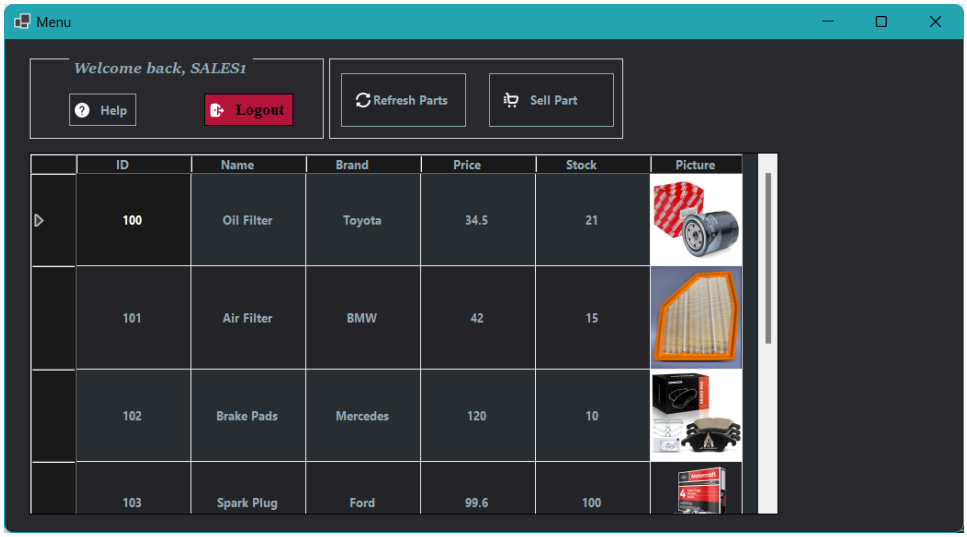
includes the Logout and Help buttons. All other menu items and interface components will differ based on the user's role.

When the user presses a button, a dedicated window opens for the specific action, prompting the user to enter different data, for example: the barcode of the product, the new stock, the old user's password. If any of the fields are incorrectly completed or the input data is invalid, the system will display appropriate error messages indicating the issue. The operation process will not proceed until valid data is provided.

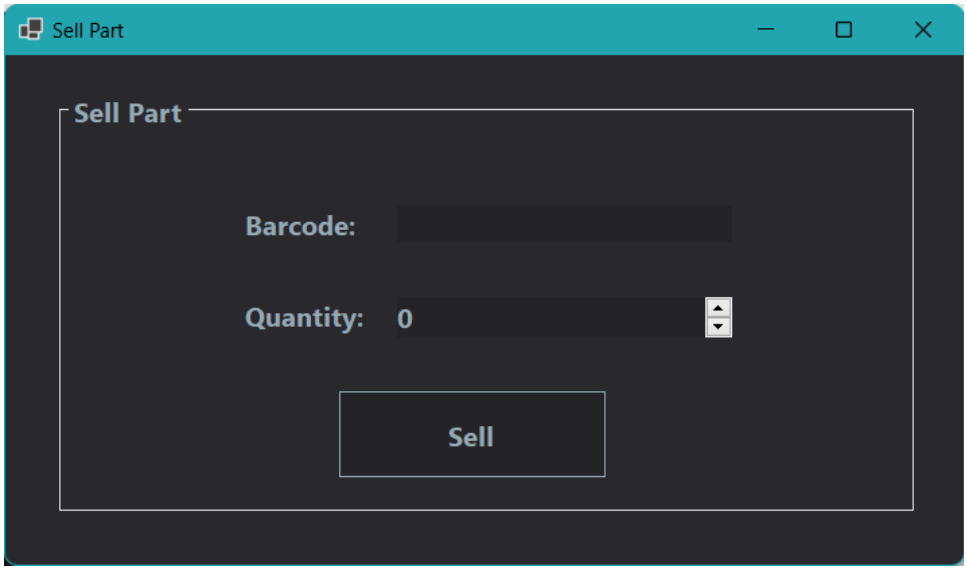


Menu interface layout

The Sales Representative has the lowest level of access, with only one role-specific action: selling products. A complete overview of all available parts is provided, allowing the user to easily view each part's ID, name, brand, price, stock level, and associated image.

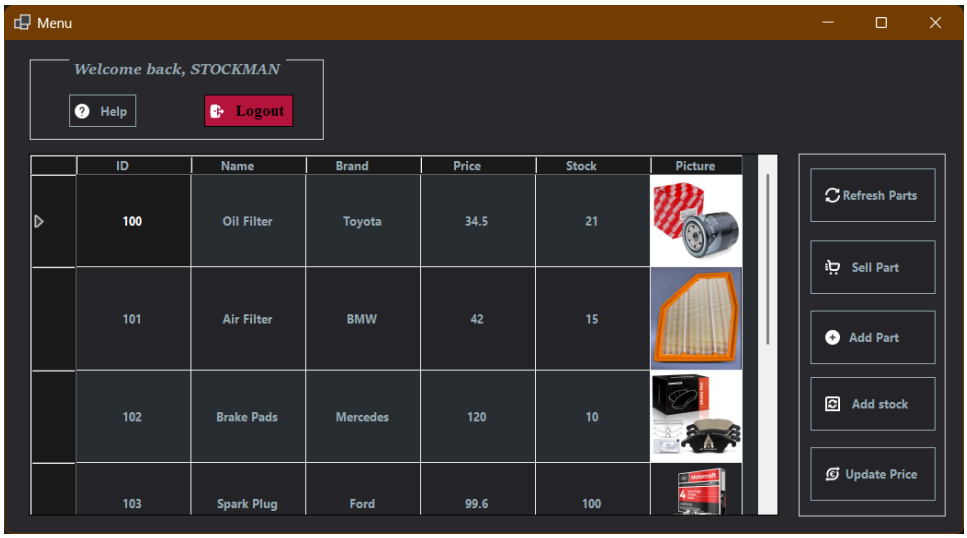


Salesman menu interface

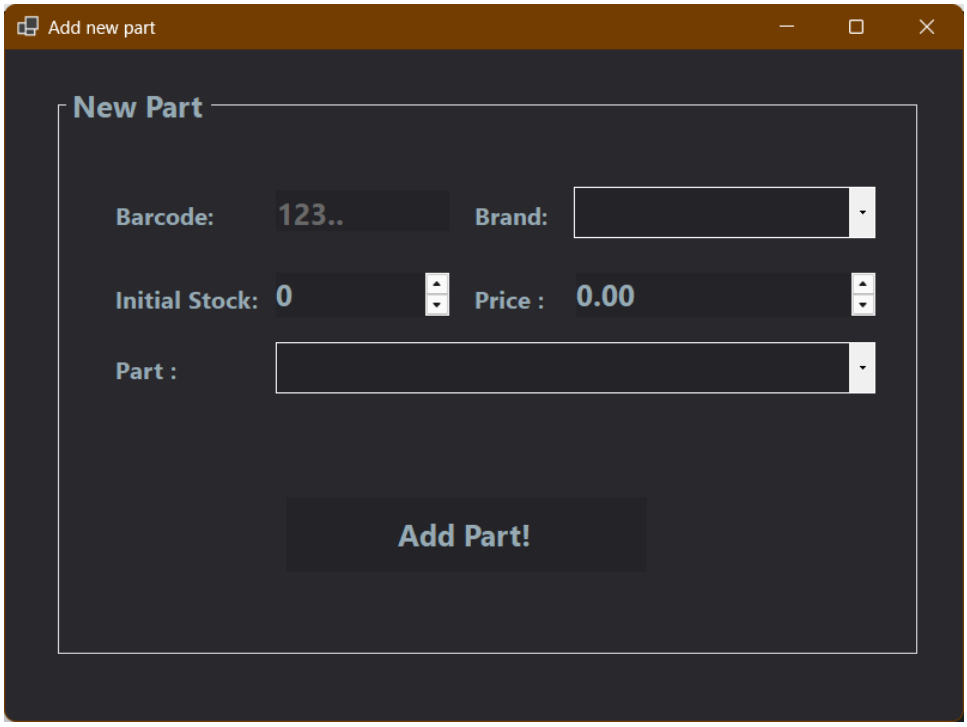


Salesman sell part interface

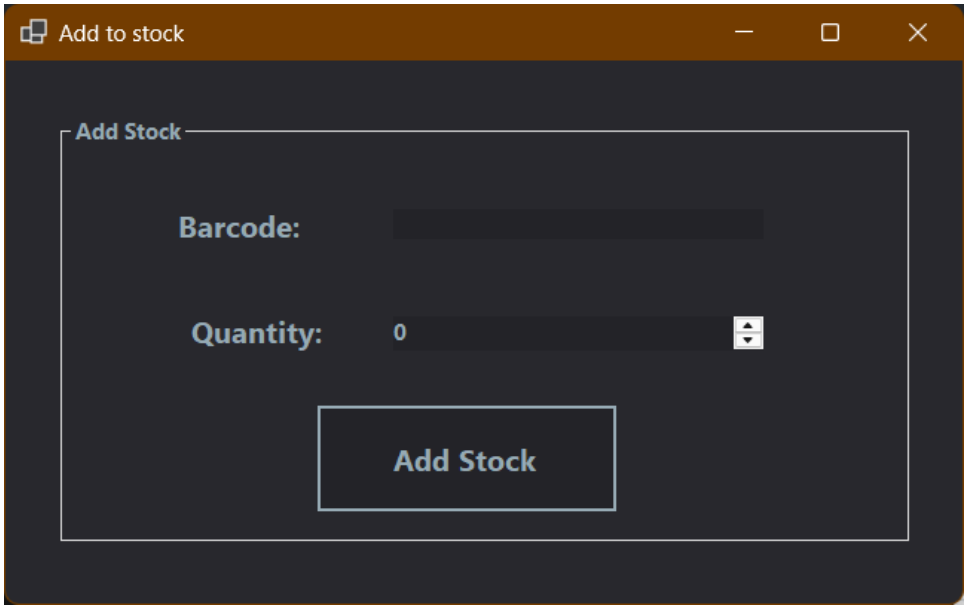
The Stock Manager has a higher level of access compared to the Sales Representative and is also responsible for managing the inventory of auto parts. Besides selling, this user can add new parts to the system, update the stock levels of existing ones and modify the price of any part currently in the inventory. The parts overview element is also present.



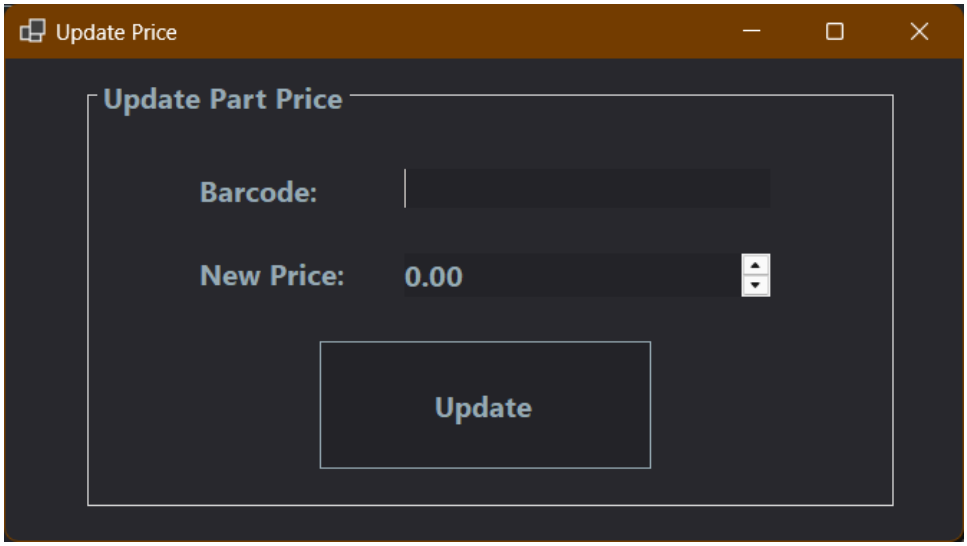
Stock Manager menu interface



Stock Manager add new part interface

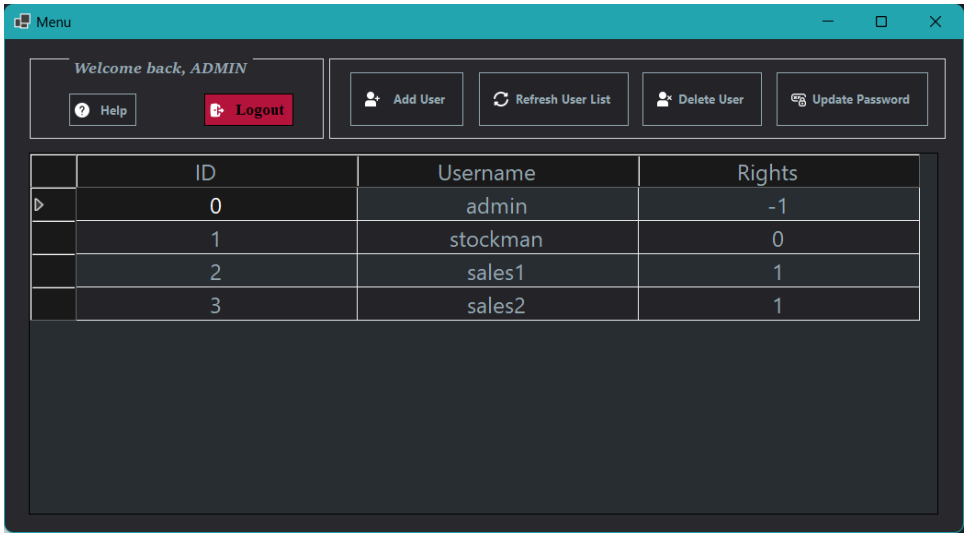


Stock Manager add to stock interface

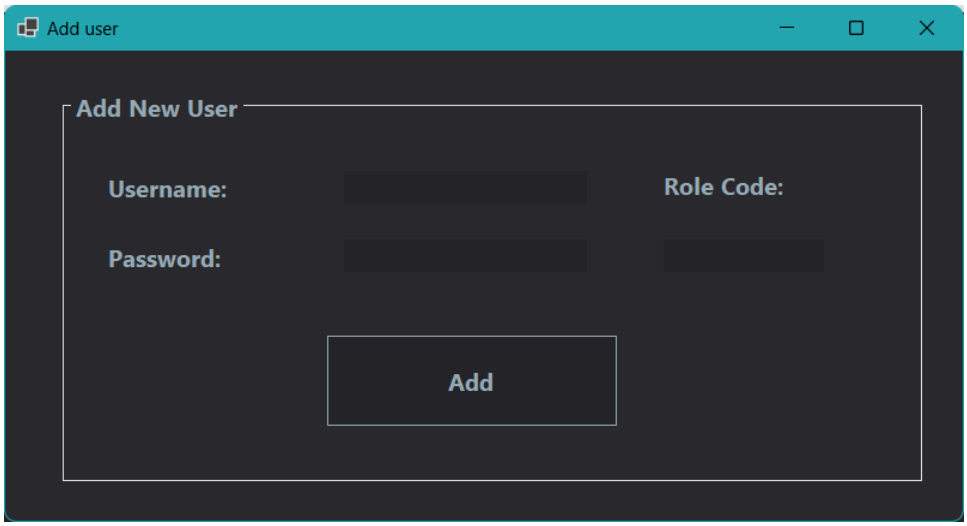


Stock Manager update part price interface

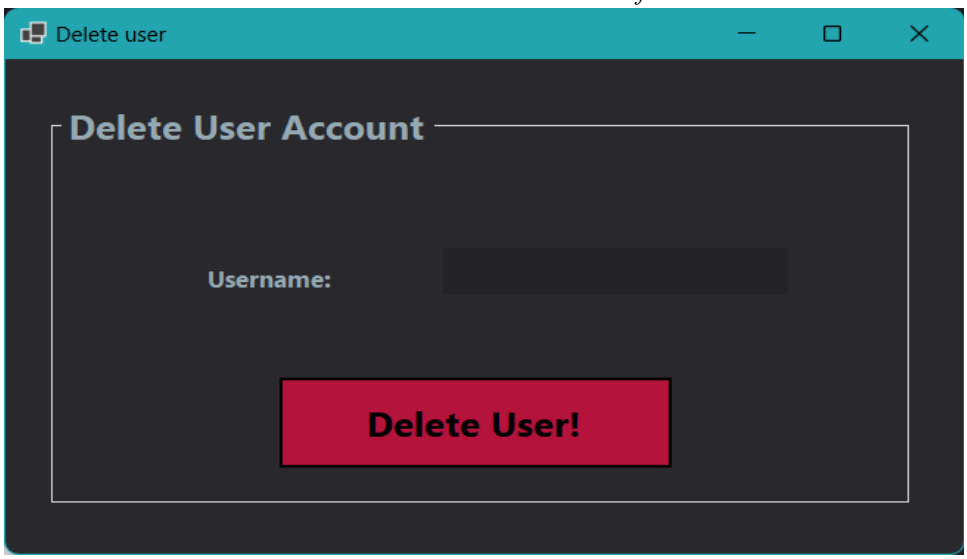
The Administrator has the highest level of access in the system and full control over user management. Upon logging in, the admin is greeted with an interface that includes options such as adding a new user, listing all existing users, deleting user accounts, and updating passwords. This role also has visibility over all usernames, their assigned rights, and unique user IDs. Through this interface, the admin can create new accounts for stock managers or sales representatives, assign appropriate access rights, and maintain overall system security and organization.



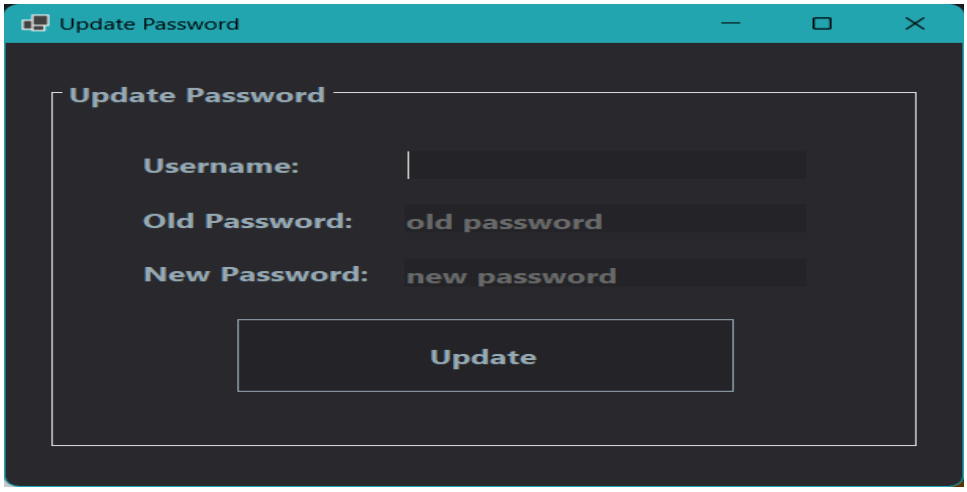
Administrator menu interface



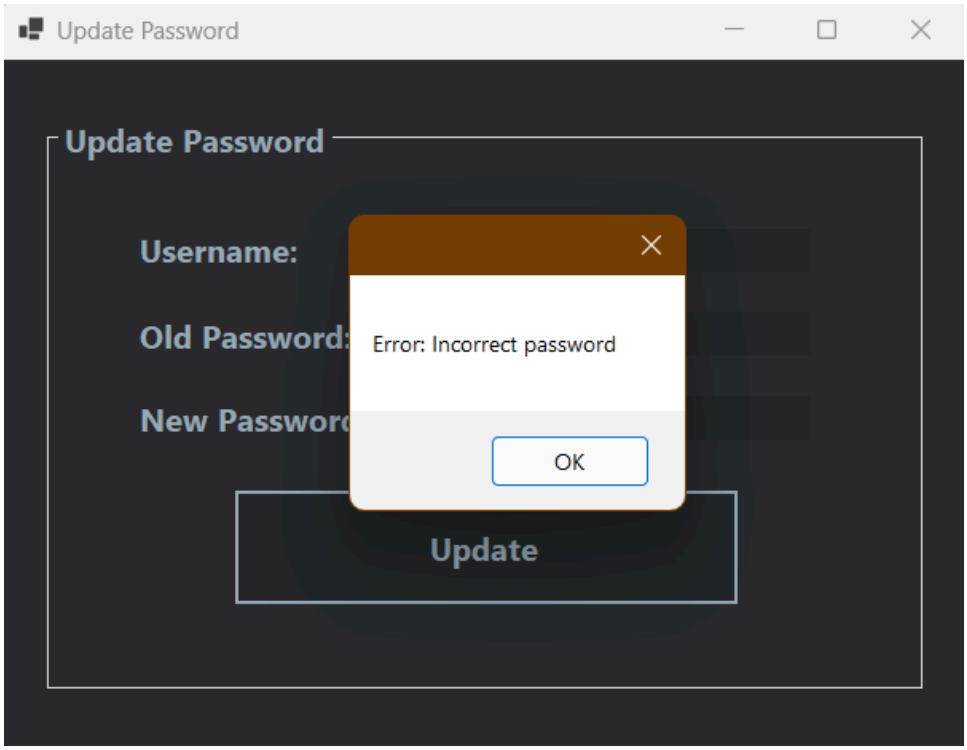
Administrator add new user interface



Administrator delete user interface



Administrator update user's password interface



An example of an error message displayed

3.2 Hardware Interfaces

The primary hardware interface is with the keyboard and mouse, which are used for standard navigation and data entry across all user roles, as well as the display monitor to visualize the application. There are no special commands for the mouse and keyboard, the control being intuitive and minimalistic.

No direct communication with low-level hardware components is required. It relies on the operating system to handle all device interactions and no specialized communication protocol is implemented within the application.

3.3 Software Interfaces

In order for the application to work, the user will need a computer running Windows operating system and .NET 8.0 installed. The application stores all its data locally in a SQLite database, which does not require a separate database server or any additional tools.

3.4 Communications Interfaces

There is no need for any communication protocol to be established over email, the internet, or any external network. The application operates entirely offline, with all functionalities and data stored and processed locally.

4. System Features

4.1 Sell auto parts

4.1.1 Description and Priority

The feature allows a Sales Representative or a Stock Manager to sell auto parts. The system shall verify for availability in stock and complete the sale only if the data is valid. This is a core functionality of the application.
Priority: High

4.1.2 Stimulus/Response Sequences

User Action: The Sales Representative or the Stock Manager selects the "Sell Part" button from the menu.
System Response: A window opens requesting a barcode and quantity.

User Action: The Sales Representative or the Stock Manager enters the barcode and quantity, then selects "Sell".
System Response: The system validates the input. If valid, it sells the product, updates the database and displays a confirmation message. If the input is invalid, an error message is shown.

4.1.3 Functional Requirements

REQ-1: The system displays a form where the Sales Representative can enter the part's barcode and quantity.
REQ-2: The system shall verify if the barcode exists in the database.
REQ-3: The system shall verify if there is enough stock for the requested quantity.
REQ-4: If the part is not found or the stock is too low, the system displays a clear error message.
REQ-5: If everything is valid, the system saves the sale and updates the stock.
REQ-6: If any input is missing or invalid, the system displays a clear error message and the sale will not go through.

4.2 Manage Parts Inventory

4.2.1 Description and Priority

This feature allows the Stock Manager to handle inventory-related tasks. The user can add new parts, increase the quantity of existing parts and modify prices. This feature ensures that the product database is accurate and up to date.
Priority: High

4.2.2 Stimulus/Response Sequences

User Action: The Stock Manager selects “Add Product” from the inventory menu.
System Response: The system displays a form for entering product details: name, barcode, brand, price and initial quantity.

User Action: The Stock Manager fills in all required fields and clicks “Add”
System Response: The system validates the input. If valid, it adds the new product to the database and displays a confirmation message. If the input is invalid, an error message is shown.

User Action: The Stock Manager clicks on “Add to Stock”
System Response: The system displays a field to enter the barcode of the part and a new quantity to be added.

User Action: The Stock Manager enters the quantity and clicks “Add”
System Response: The system updates the stock level of the selected product and displays a confirmation message. If the input is invalid, an error message is shown.

User Action: The Stock Manager selects “Modify Price”
System Response: The system displays a field to enter the barcode of the part and a field to enter a new price.

User Action: The Stock Manager enters the new price and clicks “Update”
System Response: The system saves the new price and displays a confirmation message. If the input is invalid, an error message is shown.

4.2.3 Functional Requirements

REQ-1: The system allows the Stock Manager to add new products by entering the name, barcode, brand, starting stock, and price.
REQ-2: The system shall verify that all required fields are filled in before saving a new product.
REQ-3: The system allows the Stock Manager to choose an existing product and add more stock to it.
REQ-4: After confirming, the system updates the product’s stock amount.
REQ-5: The system allows the Stock Manager to change the price of a product.
REQ-6: The new price is saved and a confirmation message is shown.
REQ-7: If any input is missing or invalid, the system displays a clear error message and stops the operation.

4.3 Manage Users and Roles

4.3.1 Description and Priority

This feature allows the Administrator to create new accounts for stock managers or sales representatives, remove users or change their passwords. This feature helps maintain system control and secure access.
Priority: High

4.3.2 Stimulus/Response Sequences

User Action: The Administrator selects “Add User” from the menu.
System Response: The system displays a form to enter a username, password, and select a role.

User Action: The Administrator fills in the fields and selects “Add”
System Response: The system shall verify the input. If valid, the user is added and a success message appears. If not, an error message is shown.

User Action: The Administrator clicks on “Delete User” from the menu.
System Response: The system displays a form to enter the username of the user to be deleted.

User Action: The Administrator enters the username and selects “Delete”
System Response: The system shall verify the input. If valid, the user is deleted. If not, an error message is shown.

User Action: The Administrator clicks on “Reset Password” from the menu
System Response: The system displays a form to enter the username of the user, the current password, and a new password.

User Action: The Administrator fills in the fields and clicks “Update”
System Response: The system shall verify the input. If valid, the user’s password is updated. If not, an error message is shown.

4.3.3 Functional Requirements

REQ-1: The system allows the Administrator to create a new user by entering a username, password, and selecting a role.
REQ-2: The system shall verify that all required fields are filled in before saving the new user.
REQ-3: The system allows the Administrator to see a list of all users, including their usernames and roles.
REQ-4: The system allows the Administrator to delete a user.
REQ-5: The system allows the Administrator to change a user’s password.
REQ-6: If any input is missing or invalid, the system displays a clear error message and stops the operation.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The application must be able to run on Windows 10 or later, using the .NET 8 runtime. Due to its use of a local access database and lightweight implementation, a system with at least 4 GB of RAM and a quad-core CPU is expected to provide optimal performance.

5.2 Safety Requirements

There are no safety requirements regarding our software application.

5.3 Security Requirements

As the application does not use any external network communication and all data is stored locally in a database on the host machine, the risk of data corruption or theft due to remote attacks is minimized. The primary security concern remains physical or direct access to the computer, such as brute force attacks targeting user accounts and passwords, or local database exploits if access is obtained.

To protect user information, the application does not store passwords in plain text. Instead, it uses a secure hashing method to store only hashed versions of the passwords. This way, even if someone gains access to the database, they cannot see or recover the actual passwords.

5.4 Software Quality Attributes

Usability:

The application is designed to be intuitive and easy to use, requiring no prior technical knowledge from the end-user. Common operations, such as selling a part, follow a straightforward and logical workflow that can be completed with ease. A help document is also provided.

Maintainability:

The source code follows general industry-standard coding conventions to ensure readability and consistency. Each functional module (e.g., database management, exceptions) is implemented as a separate component to provide ease of maintenance. This structure allows future developers to update or replace individual parts of the application without affecting unrelated functionality.

Reliability:

The application runs reliably during normal use and has been tested to complete all main operations without crashes or data loss.

Portability:

The application must run as is on any Windows 10 or later system that has .NET 8 installed. The application is deployed as a standalone solution, consisting of a single executable file and a local database file.

Adaptability:

The architecture should support replacing the local database with a remote SQL Server or cloud database in the future with no changes to business logic code.

Testability:

Unit tests have been implemented for the core functionalities of the application, focusing on the database operations and the main features of the auto parts management system. These tests ensure the correctness and stability of the application's critical components during development and future updates.

Robustness:

The application handles invalid inputs (e.g., wrong part ID) by displaying appropriate error messages and not crashing.

5.5 Business Rules

The application defines three user roles: Administrator, Stock Manager, and Sales Representative. Role-based access restrictions apply throughout the system, with each user role providing access to a distinct set of features:

- The Administrator is responsible for managing user accounts, with permissions to create or delete users and update existing passwords.
- The Stock Manager is authorized to sell parts and to manage the inventory, including adding new parts, updating stock levels, and modifying prices.
- The Sales Representative is performing the product sales, without access to inventory or user management functions.

6. Unit Testing

The core modules of the application — including the database manager and the action handler responsible for all application actions — were tested to verify their complete functionality. To avoid issues related to concurrent access, the tests were performed using an in-memory database.

6.1 Purpose of Testing

Unit tests were conducted to verify the functionality of each module in isolation, ensuring that all critical components of the application behave according to the specifications defined in the SRS document.

6.2 Testing Strategy

A white-box testing strategy was adopted, focusing on the internal logic of the modules and covering relevant execution paths. To eliminate the influence of external factors, an in-memory database and an isolated test environment were used.

6.3 Types of Tests Performed

Unit tests were performed for individual functions (e.g., data validation, action handling).

6.4 Tools and Technologies

The tests were implemented using MSTest, the official unit testing framework for .NET, and test cases were executed using Visual Studio's Test Explorer.

6.5 Passing Criteria

A test is considered successful if:

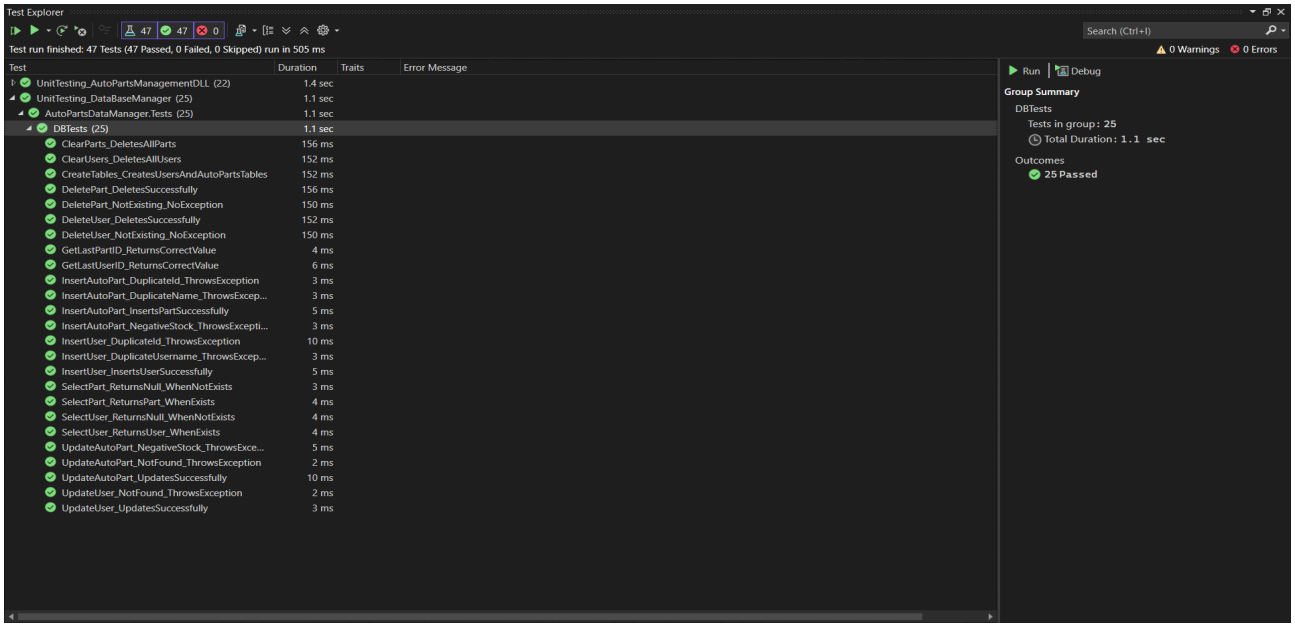
- It returns the expected result for valid inputs
- It throws the correct exceptions or error codes for invalid inputs

6.6 Overall Results

All essential unit tests were executed successfully. A complete overview of all the test cases can be found in the tables below.

Test Method	Purpose	Expected Outcome	Result
CreateTables_CreatesUsersAndAutoPartsTables	Verify table creation	Two empty tables are created	PASS
InsertUser_InsertsUserSuccessfully	Insert a new user	User appears in user list	PASS
InsertUser_DuplicateId_ThrowsException	Insert user with duplicate ID	Throws ConstraintViolatedException	PASS
InsertUser_DuplicateUsername_ThrowsException	Insert user with duplicate username	Throws ConstraintViolatedException	PASS
InsertAutoPart_InsertsPartSuccessfully	Insert a new part	Part appears in parts list	PASS
InsertAutoPart_NegativeStock_ThrowsException	Insert part with negative stock	Throws InvalidStockException	PASS
InsertAutoPart_DuplicateId_ThrowsException	Insert part with duplicate ID	Throws ConstraintViolatedException	PASS
InsertAutoPart_DuplicateName_ThrowsException	Insert part with duplicate name	Throws ConstraintViolatedException	PASS
SelectUser_ReturnsUser_WhenExists	Select existing user	Returns correct user	PASS
SelectUser_ReturnsNull_WhenNotExists	Select non-existent user	Returns null	PASS
SelectPart_ReturnsPart_WhenExists	Select existing part	Returns correct part	PASS
SelectPart_ReturnsNull_WhenNotExists	Select non-existent part	Returns null	PASS
UpdateUser_UpdatesSuccessfully	Update user data	Changes are saved	PASS
UpdateUser_NotFound_ThrowsException	Update non-existent user	Throws RecordNotFoundException	PASS
UpdateAutoPart_UpdatesSuccessfully	Update part data	Changes are saved	PASS
UpdateAutoPart_NegativeStock_ThrowsException	Update part to negative stock	Throws InvalidStockException	PASS
UpdateAutoPart_NotFound_ThrowsException	Update non-existent part	Throws RecordNotFoundException	PASS
DeleteUser_DeletesSuccessfully	Delete existing user	User is removed	PASS
DeleteUser_NotExisting_NoException	Delete non-existent user	No exception thrown	PASS
DeletePart_DeletesSuccessfully	Delete existing part	Part is removed	PASS
DeletePart_NotExisting_NoException	Delete non-existent part	No exception thrown	PASS
ClearUsers_DeletesAllUsers	Clear user table	All users removed	PASS
ClearParts_DeletesAllParts	Clear parts table	All parts removed	PASS
GetLastUserID_ReturnsCorrectValue	Get last user ID	Returns highest user ID	PASS
GetLastPartID_ReturnsCorrectValue	Get last part ID	Returns highest part ID	PASS

Database Tests

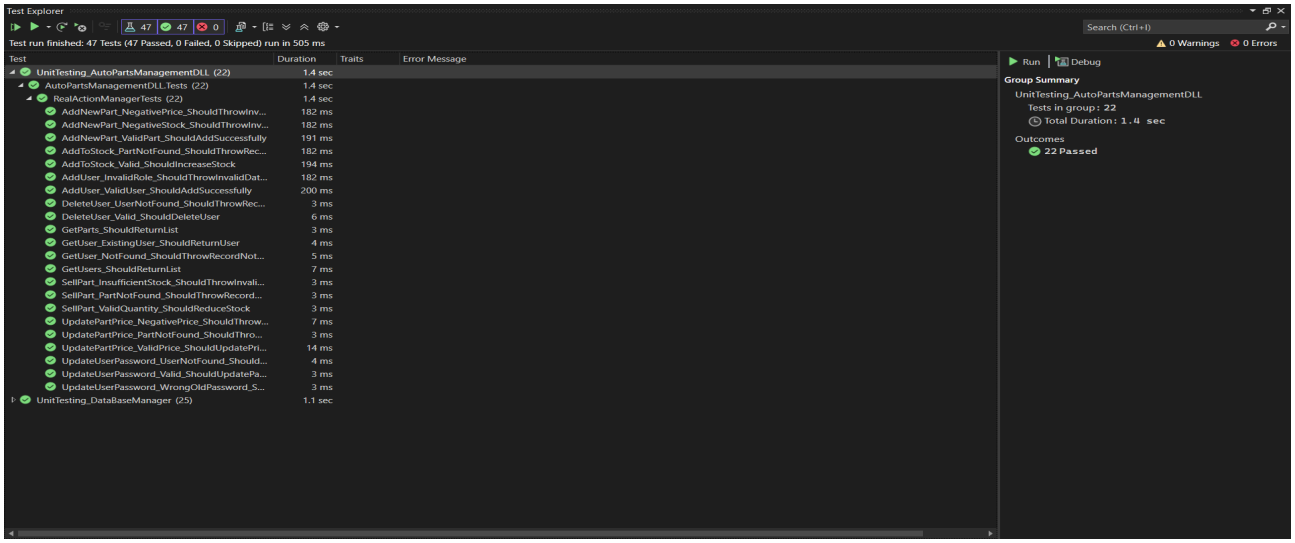


Test Method	Purpose	Expected Outcome	Result
AddNewPart_ValidPart_ShouldAddSuccessfully	Add a valid part	Part is saved	PASS
AddNewPart_NegativeStock_ShouldThrowInvalidStockException	Add part with negative stock	Throws InvalidStockException	PASS
AddNewPart_NegativePrice_ShouldThrowInvalidDataException	Add part with negative price	Throws InvalidDataException	PASS
SellPart_ValidQuantity_ShouldReduceStock	Sell part with valid quantity	Stock decreases	PASS
SellPart_InsufficientStock_ShouldThrowInvalidStockException	Sell part with insufficient stock	Throws InvalidStockException	PASS
SellPart_PartNotFound_ShouldThrowRecordNotFoundException	Sell non-existent part	Throws RecordNotFoundException	PASS
AddToStock_Valid_ShouldIncreaseStock	Add stock to existing part	Stock increases	PASS
AddToStock_PartNotFound_ShouldThrowRecordNotFoundException	Add stock to non-existent part	Throws RecordNotFoundException	PASS
UpdatePartPrice_ValidPrice_ShouldUpdatePrice	Update price of a part	Price is updated	PASS
UpdatePartPrice_NegativePrice_ShouldThrowInvalidDataException	Set negative price	Throws InvalidDataException	PASS
UpdatePartPrice_PartNotFound_ShouldThrowRecordNotFoundException	Update price of non-existent part	Throws RecordNotFoundException	PASS
GetParts_ShouldReturnList	Retrieve parts list	Returns non-empty list	PASS

Action Manager - Part Management Tests

Test Method	Purpose	Expected Outcome	Result
AddUser_ValidUser_ShouldAddSuccessfully	Add valid user	User is added	PASS
AddUser_InvalidRole_ShouldThrowInvalidDataException	Add user with invalid role	Throws InvalidDataException	PASS
UpdateUserPassword_WrongOldPassword_ShouldThrowInvalidDataException	Wrong old password when changing password	Throws InvalidDataException	PASS
UpdateUserPassword_UserNotFound_ShouldThrowRecordNotFoundException	User not found	Throws RecordNotFoundException	PASS
GetUser_ExistingUser_ShouldReturnUser	Retrieve existing user	Returns user	PASS
GetUser_NotFound_ShouldThrowRecordNotFoundException	Retrieve non-existent user	Throws RecordNotFoundException	PASS
DeleteUser_Valid_ShouldDeleteUser	Delete existing user	User deleted	PASS
DeleteUser_UserNotFound_ShouldThrowRecordNotFoundException	Delete non-existent user	Throws RecordNotFoundException	PASS
GetUsers_ShouldReturnList	Retrieve users list	Returns non-empty list	PASS

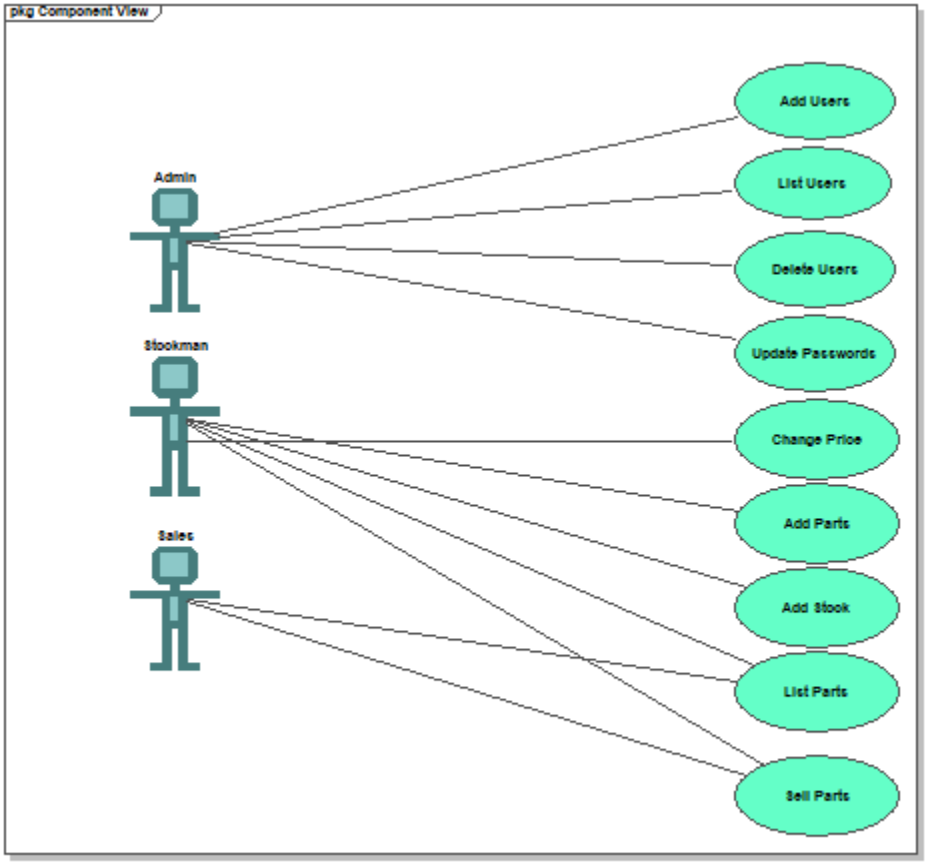
Action Manager - User Management Tests



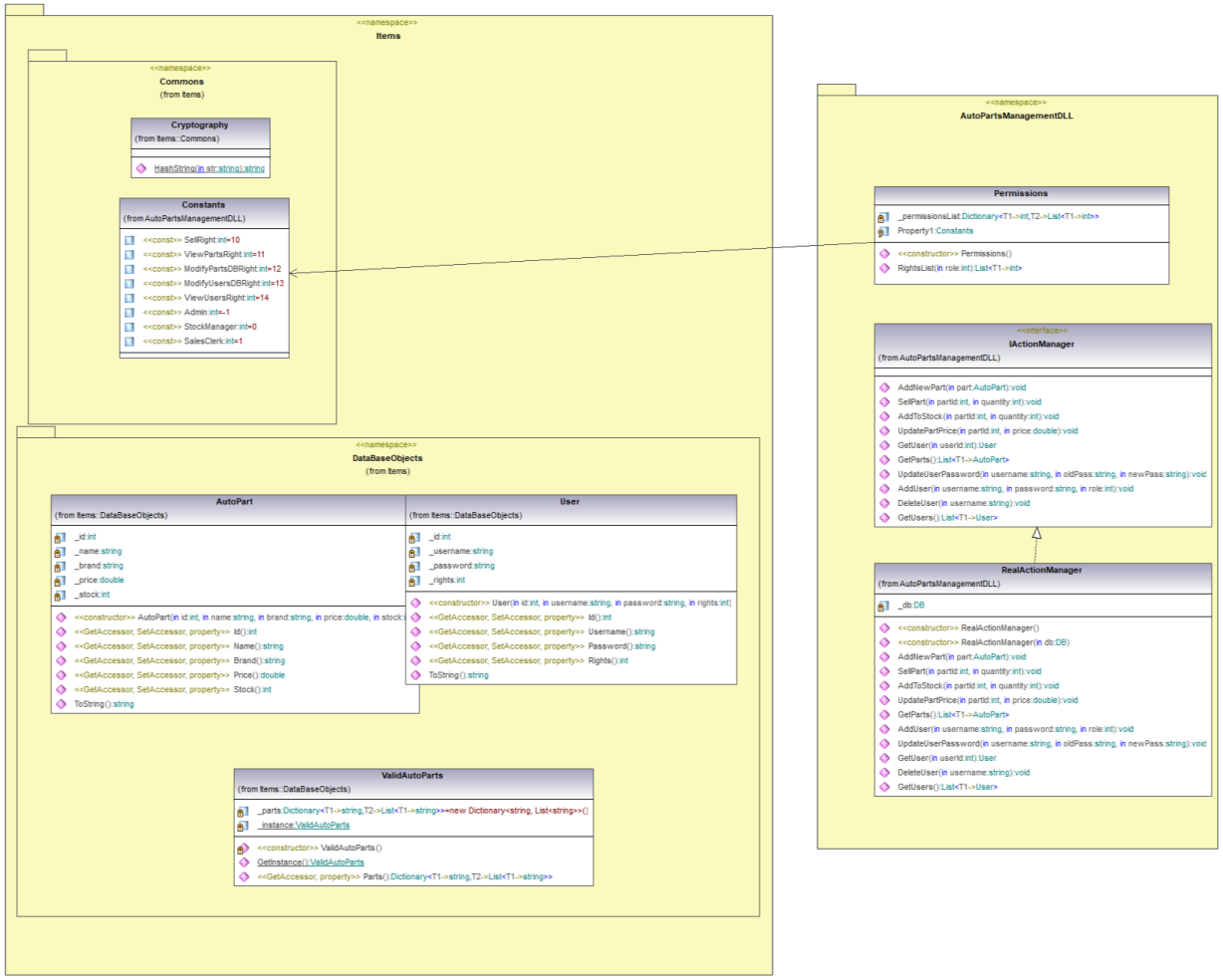
Appendix A: Glossary

N/A

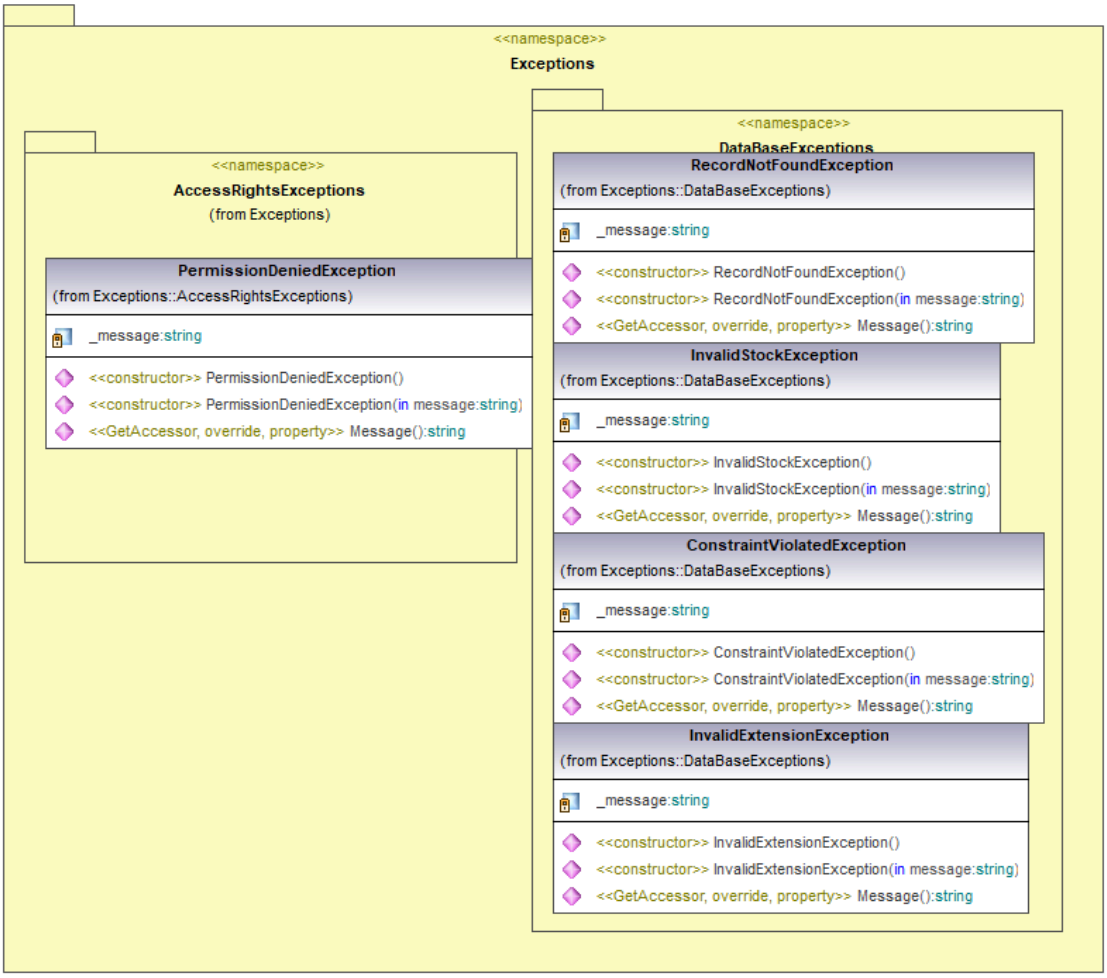
Appendix B: Analysis Models



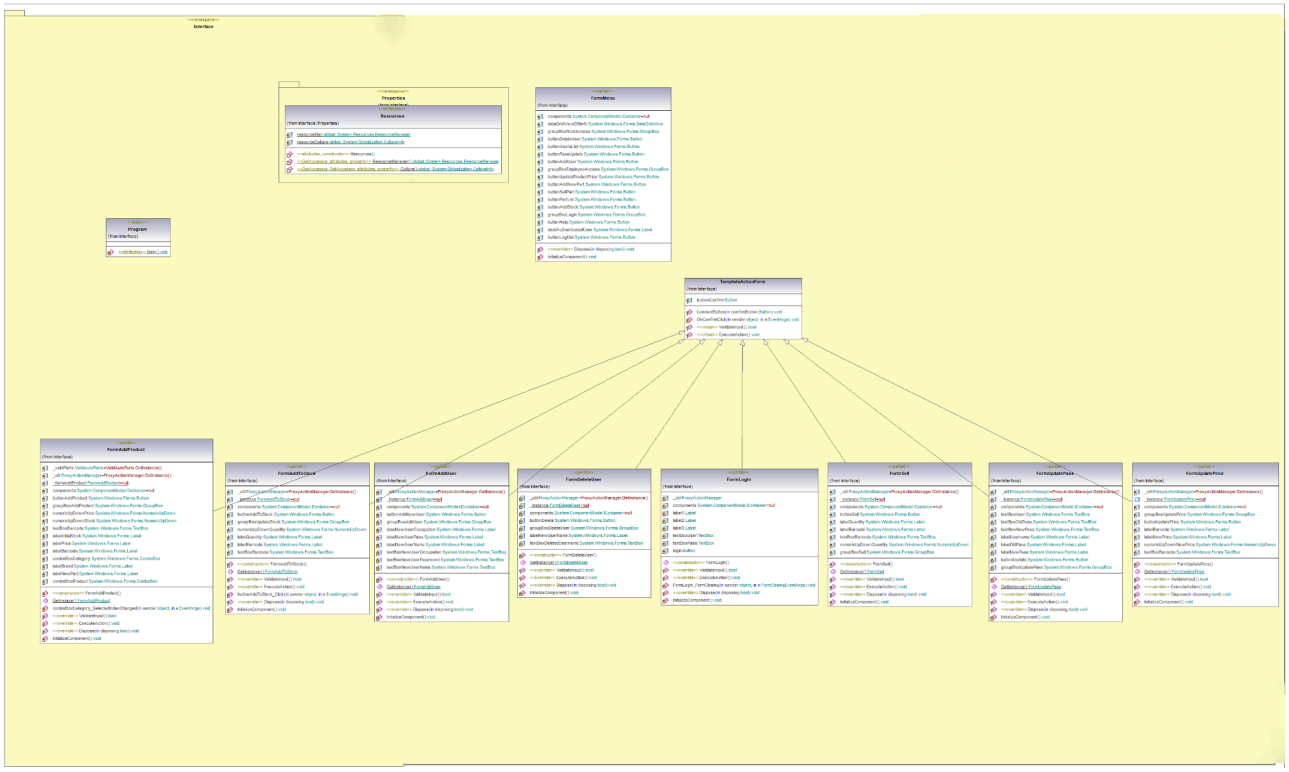
Use case diagram



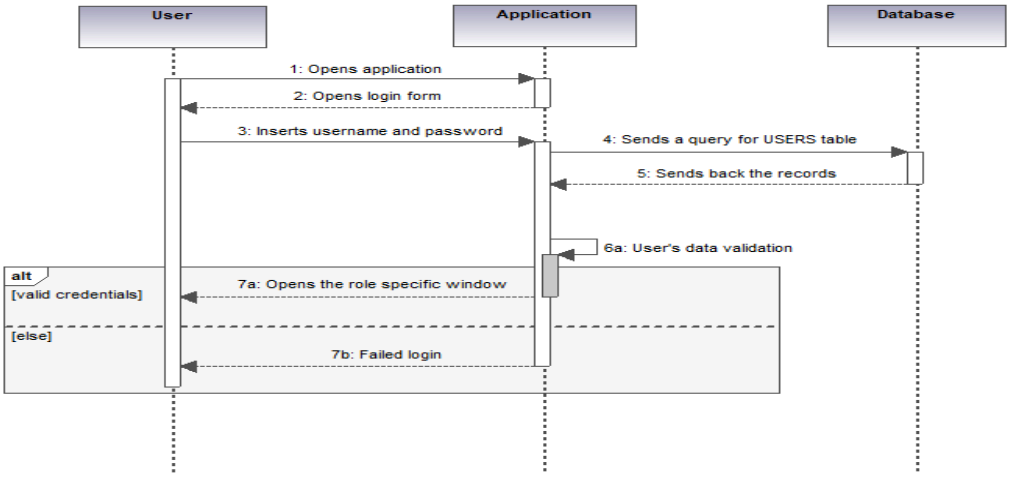
Class diagram - part 1



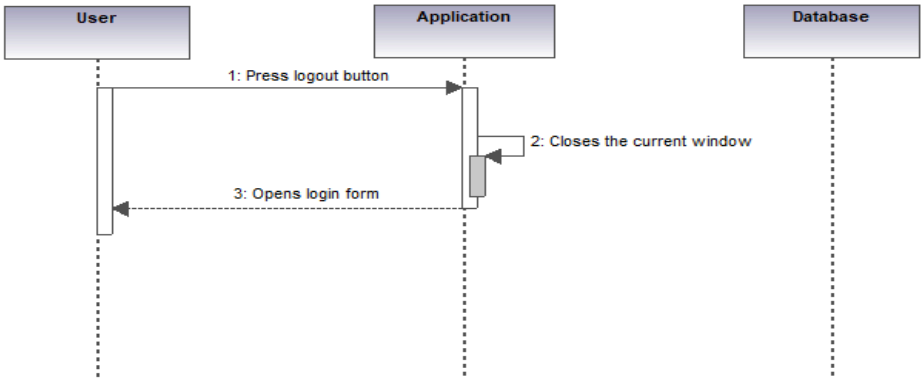
Class diagram - part 2



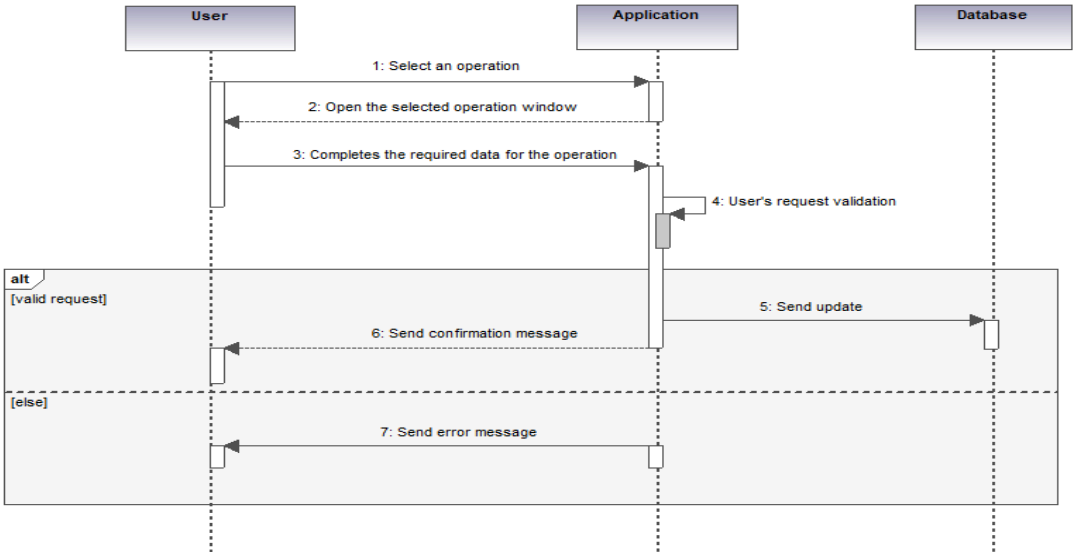
Class diagram - part 3



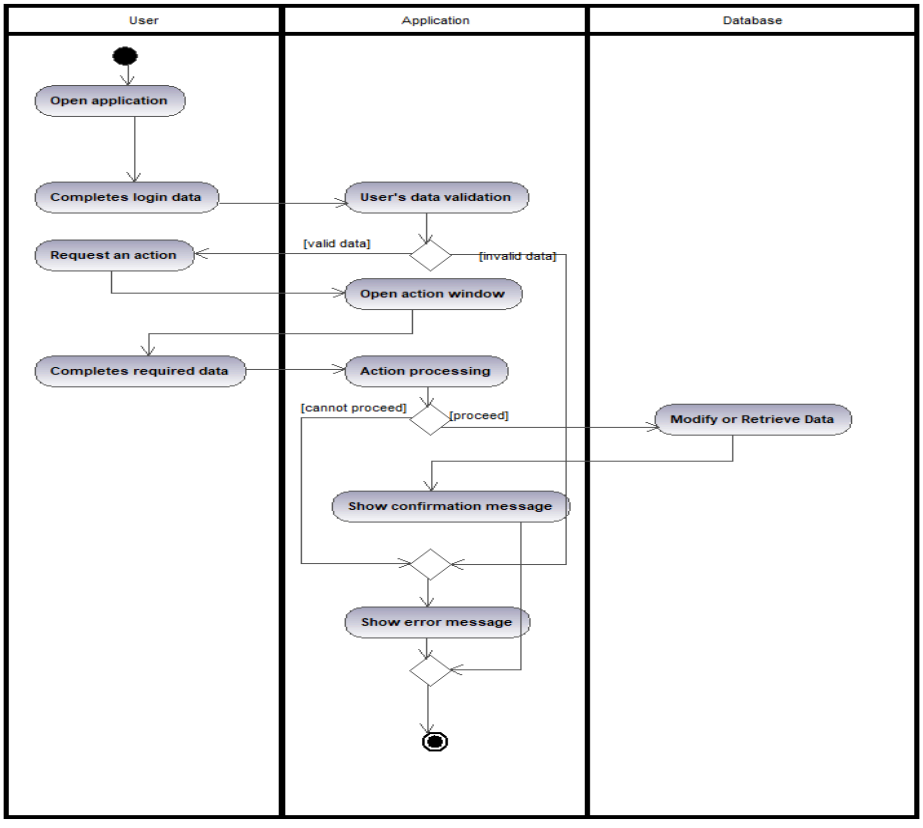
Sequence diagram of login process



Sequence diagram of logout process



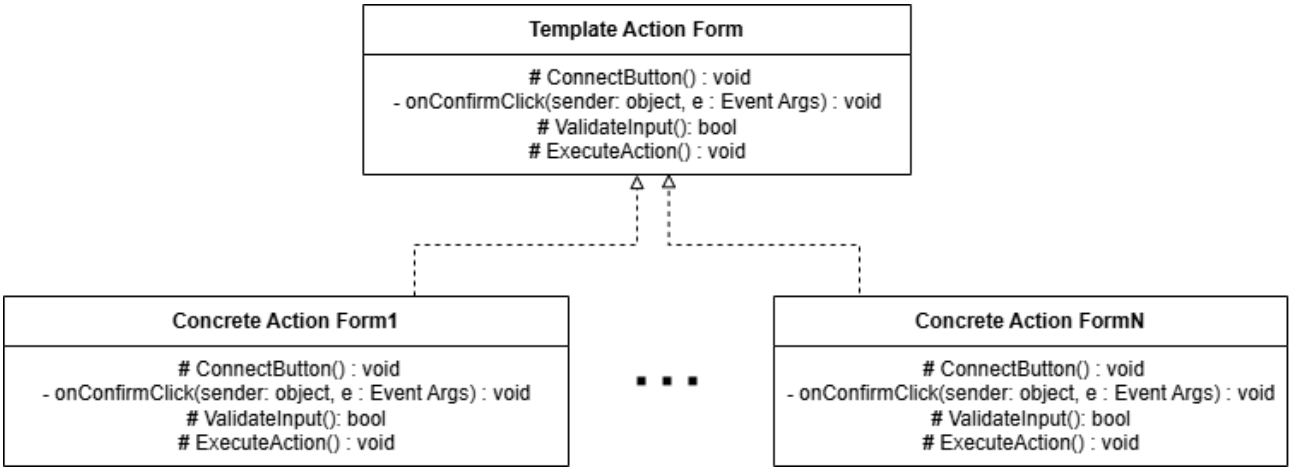
Sequence diagram of a generic user action



Activity diagram of a generic user action

Design patterns:

1. Template:



Template Form class:

```
public class TemplateActionForm : Form
{
    protected Button buttonConfirm;

    // No constructor code to create a button!
    protected void ConnectButton(Button confirmButton)
    {
        buttonConfirm = confirmButton;
        buttonConfirm.Click += OnConfirmClick;
    }

    private void OnConfirmClick(object sender, EventArgs e)
    {
        if (!ValidateInput())
        {
            MessageBox.Show("Invalid data.");
            return;
        }

        try
        {
            ExecuteAction();
        }
        catch (Exception ex)
        {
            MessageBox.Show("Error: " + ex.Message);
        }
    }

    protected virtual bool ValidateInput() { return true; }
```

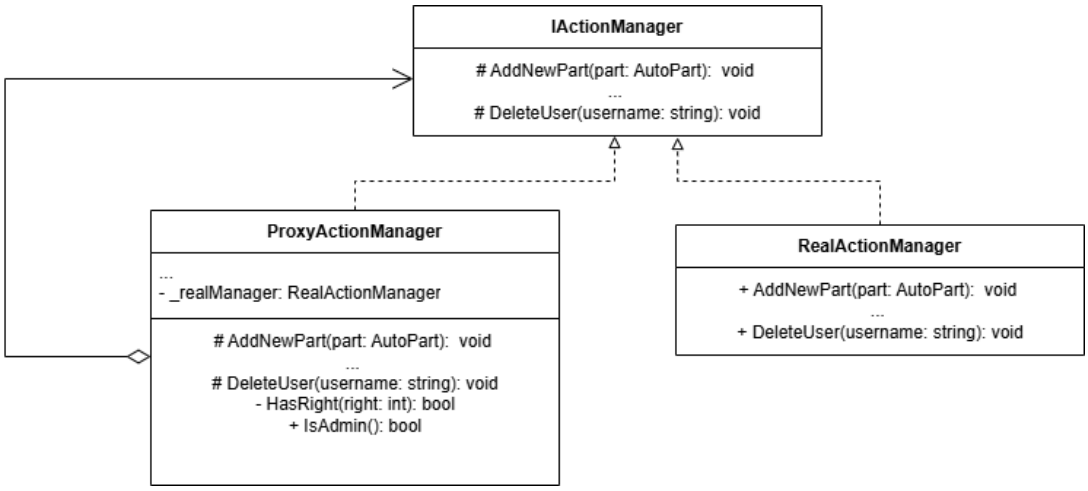
```
    protected virtual void ExecuteAction() { }
}
```

Concrete Form Class Example (Login Form):

```
public partial class FormLogIn : TemplateActionForm
{
    private ProxyActionManager _util;

    /// <summary>
    /// Initializes the login form and connects logic to the login button.
    /// </summary>
    public FormLogIn()
    {
        InitializeComponent();
        _util = ProxyActionManager.GetInstance();

        this.AcceptButton = login;
        textBoxPass.PasswordChar = '*';

        ConnectButton(login);
    }

    /// <summary>
    /// Validates input fields for username and password.
    /// </summary>
    /// <returns>True if both fields are filled; otherwise, false.</returns>
    protected override bool ValidateInput()
    {
        return !string.IsNullOrWhiteSpace(textBoxUser.Text) &&
               !string.IsNullOrWhiteSpace(textBoxPass.Text);
    }

    /// <summary>
    /// Attempts user login and redirects to menu if successful.
    /// </summary>
    protected override void ExecuteAction()
    {
        string username = textBoxUser.Text;
        string password = textBoxPass.Text;

        if (_util.Login(username, password))
        {
            Form pagina2 = new FormMenu();
            pagina2.Show();
            this.Hide();
        }
        else
        {
            MessageBox.Show("Wrong Username or Password", "Error");
            textBoxUser.Clear();
```

```
            textBoxPass.Clear();
        }
    }

    /// <summary>
    /// Handles application exit when the login form is closed.
    /// </summary>
    private void FormLogIn_FormClosing(object sender, FormClosingEventArgs e)
    {
        Application.Exit();
    }
}
```

2. Proxy



IActionManager:

```
/// <summary>
/// Interface for defining user and part management operations.
/// </summary>
public interface IActionManager
{
    /// <summary>
    /// Adds a new auto part.
    /// </summary>
    /// <param name="part">Part to be added.</param>
    void AddNewPart(AutoPart part);

    /// <summary>
    /// Processes a part sale by decreasing stock.
    /// </summary>
    /// <param name="partId">ID of the part.</param>
    /// <param name="quantity">Quantity to sell.</param>
    void SellPart(int partId, int quantity);
```

```
    /// <summary>
    /// Adds stock to an existing part.
    /// </summary>
    /// <param name="partId">ID of the part.</param>
    /// <param name="quantity">Quantity to add.</param>
    void AddToStock(int partId, int quantity);

    /// <summary>
    /// Updates the price of a specific part.
    /// </summary>
    /// <param name="partId">ID of the part.</param>
    /// <param name="price">New price value.</param>
    void UpdatePartPrice(int partId, double price);

    /// <summary>
    /// Returns the list of all auto parts.
    /// </summary>
    /// <returns>List of parts.</returns>
    List<AutoPart> GetParts();

    /// <summary>
    /// Adds a new user to the system.
    /// </summary>
    /// <param name="username">Username.</param>
    /// <param name="password">Password.</param>
    /// <param name="role">Role of the user.</param>
    void AddUser(string username, string password, int role);

    /// <summary>
    /// Updates a user's password.
    /// </summary>
    /// <param name="username">Username.</param>
    /// <param name="oldPass">Current password.</param>
    /// <param name="newPass">New password.</param>
    void UpdateUserPassword(string username, string oldPass, string
newPass);

    /// <summary>
    /// Retrieves a user by ID.
    /// </summary>
    /// <param name="userId">ID of the user.</param>
    /// <returns>User instance.</returns>
    User GetUser(int userId);

    /// <summary>
    /// Deletes a user by username.
    /// </summary>
    /// <param name="username">Username of the user to delete.</param>
    void DeleteUser(string username);
```

```
    /// <summary>
    /// Returns the list of all users.
    /// </summary>
    /// <returns>List of users.</returns>
    List<User> GetUsers();
}
```

Proxy Action Manager:

```
    /// <summary>
    /// Provides controlled access to application actions using permission checks.
    /// Acts as a proxy to the RealActionManager.
    /// </summary>
    public class ProxyActionManager : IActionManager
    {
        private static ProxyActionManager _instance = null;
        private DB _db;
        private RealActionManager _realManager;
        private User _currentUser;
        private Permissions _permissions;

        /// <summary>
        /// Private constructor initializes dependencies.
        /// </summary>
        private ProxyActionManager()
        {
            _db = DB.GetInstance("AutoParts.db");
            _realManager = new RealActionManager();
            _permissions = new Permissions();
        }

        /// <summary>
        /// Gets the singleton instance of ProxyActionManager.
        /// </summary>
        public static ProxyActionManager GetInstance()
        {
            if (_instance == null)
            {
                _instance = new ProxyActionManager();
            }
            return _instance;
        }

        /// <summary>
        /// Gets the currently logged-in user.
        /// </summary>
        public User CurrentUser => _currentUser;
```

```
        /// <summary>
        /// Attempts to log in with a given username and password.
        /// </summary>
        /// <param name="username">Username to log in.</param>
        /// <param name="password">Password to check.</param>
        /// <returns>True if login is successful; otherwise, false.</returns>
        public bool Login(string username, string password)
        {
            List<User> users = _db.SelectAllUsers();
            string hashedPassword = Cryptography.HashString(password);

            for (int i = 0; i < users.Count; i++)
            {
                if (users[i].Username == username && users[i].Password ==
hashedPassword)
                {
                    _currentUser = users[i];
                    return true;
                }
            }

            return false;
        }

        /// <summary>
        /// Adds a new auto part if the user has permission.
        /// </summary>
        public void AddNewPart(AutoPart part)
        {
            if (!HasRight(Constants.ModifyPartsDBRight))
                throw new PermissionDeniedException();

            _realManager.AddNewPart(part);
        }

        /// <summary>
        /// Processes a sale for the given part and quantity.
        /// </summary>
        public void SellPart(int partId, int quantity)
        {
            if (!HasRight(Constants.SellRight))
                throw new PermissionDeniedException();

            _realManager.SellPart(partId, quantity);
        }

        /// <summary>
        /// Adds stock to an existing part.
        /// </summary>
        public void AddToStock(int partId, int quantity)
        {
```

```
            if (!HasRight(Constants.ModifyPartsDBRight))
                throw new PermissionDeniedException();

            _realManager.AddToStock(partId, quantity);
        }

        /// <summary>
        /// Updates the price of an auto part.
        /// </summary>
        public void UpdatePartPrice(int partId, double price)
        {
            if (!HasRight(Constants.ModifyPartsDBRight))
                throw new PermissionDeniedException();

            _realManager.UpdatePartPrice(partId, price);
        }

        /// <summary>
        /// Retrieves the list of all parts.
        /// </summary>
        public List<AutoPart> GetParts()
        {
            if (!HasRight(Constants.ViewPartsRight))
                throw new PermissionDeniedException();

            return _realManager.GetParts();
        }

        /// <summary>
        /// Adds a new user with specified credentials and role.
        /// </summary>
        public void AddUser(string username, string password, int role)
        {
            if (!HasRight(Constants.ModifyUsersDBRight))
                throw new PermissionDeniedException();

            _realManager.AddUser(username, password, role);
        }

        /// <summary>
        /// Updates the password of an existing user.
        /// </summary>
        public void UpdateUserPassword(string username, string oldPass, string
newPass)
        {
            if (!HasRight(Constants.ModifyUsersDBRight))
                throw new PermissionDeniedException();

            _realManager.UpdateUserPassword(username, oldPass, newPass);
        }

        /// <summary>
```

```
/// Retrieves a user by ID.
/// </summary>
public User GetUser(int userId)
{
    if (!HasRight(Constants.ViewUsersRight))
        throw new PermissionDeniedException();

    return _realManager.GetUser(userId);
}

/// <summary>
/// Deletes a user by username.
/// </summary>
public void DeleteUser(string username)
{
    if (!HasRight(Constants.ModifyUsersDBRight))
        throw new PermissionDeniedException();

    _realManager.DeleteUser(username);
}

/// <summary>
/// Retrieves the list of all users.
/// </summary>
public List<User> GetUsers()
{
    if (!HasRight(Constants.ViewUsersRight))
        throw new PermissionDeniedException();

    return _realManager.GetUsers();
}

/// <summary>
/// Checks if the current user is an administrator.
/// </summary>
public bool IsAdmin()
{
    return _currentUser != null && _currentUser.Rights == Constants.Admin;
}

/// <summary>
/// Checks if the current user has a specific permission right.
/// </summary>
/// <param name="right">The right to be verified.</param>
/// <returns>True if the user has the right; otherwise, false.</returns>
private bool HasRight(int right)
{
    List<int> rights = _permissions.RightsList(_currentUser.Rights);
    for (int i = 0; i < rights.Count; i++)
    {
        if (rights[i] == right)
        {
```

```
                    return true;
                }
            }
            return false;
        }
    }
```

Real Action Manager:

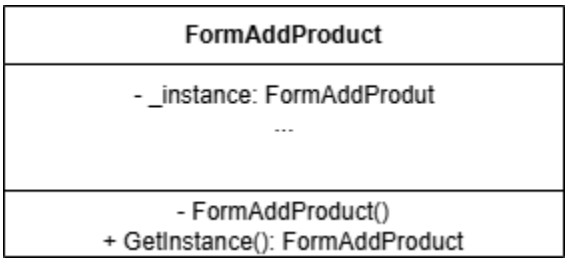
```
/// <summary>
/// Handles actual operations for managing auto parts and users.
/// Performs direct data manipulation through the database layer.
/// </summary>
public class RealActionManager : IActionManager
{
    private DB _db;

    /// <summary>
    /// Initializes a new instance using the default database file.
    /// </summary>
    public RealActionManager()
    {
        _db = DB.GetInstance("AutoParts.db");
        _db.CreateTables();
    }

    /// <summary>
    /// Initializes a new instance using a custom database instance.
    /// </summary>
    /// <param name="db">The database instance to use.</param>
    public RealActionManager(DB db)
    {
        _db = db;
        _db.CreateTables();
    }

    /// <summary>
    /// Adds a new auto part to the system.
    /// </summary>
    public void AddNewPart(AutoPart part)
    {
        if (part.Stock < 0)
            throw new InvalidStockException("Stocul nu poate fi mai mic ca 0");
        if (part.Price < 0)
            throw new InvalidDataException("Prețul trebuie să fie pozitiv.");

        _db.Insert(part);
    }

    /// <summary>
```

```
    /// Sells a part by decreasing its stock.
    /// </summary>
    public void SellPart(int partId, int quantity)
    {
        AutoPart part = _db.SelectPart(partId);
        if (part == null)
            throw new RecordNotFoundException("Piesa nu a fost găsită.");

        int newStock = part.Stock - quantity;
        if (newStock < 0)
            throw new InvalidStockException("Stoc insuficient.");

        part.Stock = newStock;
        _db.Update(part);
    }

    /// <summary>
    /// Increases the stock of a part.
    /// </summary>
    public void AddToStock(int partId, int quantity)
    {
        AutoPart part = _db.SelectPart(partId);
        if (part == null)
            throw new RecordNotFoundException("Piesa nu a fost găsită.");

        part.Stock += quantity;
        _db.Update(part);
    }

    /// <summary>
    /// Updates the price of an existing part.
    /// </summary>
    public void UpdatePartPrice(int partId, double price)
    {
        if (price < 0)
            throw new InvalidDataException("Preț invalid.");

        AutoPart part = _db.SelectPart(partId);
        if (part == null)
            throw new RecordNotFoundException("Piesa nu a fost găsită.");

        part.Price = price;
        _db.Update(part);
    }

    /// <summary>
    /// Retrieves all auto parts from the database.
    /// </summary>
    public List<AutoPart> GetParts()
    {
        return _db.SelectAllParts();
    }
```

```
    /// <summary>
    /// Adds a new user to the system.
    /// </summary>
    public void AddUser(string username, string password, int role)
    {
        if (role != Constants.StockManager && role != Constants.SalesClerk)
            throw new InvalidDataException("Rol invalid.");

        int id = _db.GetLastUserID() + 1;
        string hashed = Cryptography.HashString(password);
        User user = new User(id, username, hashed, role);
        _db.Insert(user);
    }

    /// <summary>
    /// Updates the password for an existing user.
    /// </summary>
    public void UpdateUserPassword(string username, string oldPass, string
newPass)
    {
        User user = _db.SelectUser(username);
        if (user == null)
            throw new RecordNotFoundException("Utilizatorul nu a fost găsit.");

        string oldHash = Cryptography.HashString(oldPass);
        if (user.Password != oldHash)
            throw new InvalidDataException("Parolă incorectă");

        user.Password = Cryptography.HashString(newPass);
        _db.Update(user);
    }

    /// <summary>
    /// Retrieves a user by ID.
    /// </summary>
    public User GetUser(int userId)
    {
        List<User> all = _db.SelectAllUsers();
        foreach (User u in all)
        {
            if (u.Id == userId)
                return u;
        }
        throw new RecordNotFoundException("Utilizatorul nu a fost găsit.");
    }

    /// <summary>
    /// Deletes a user by username.
    /// </summary>
    public void DeleteUser(string username)
    {
```

```
        User user = _db.SelectUser(username);
        if (user == null)
            throw new RecordNotFoundException("Utilizatorul nu a fost găsit.");
        if (user.Rights == Constants.Admin)
            throw new PermissionDeniedException("Admin nu poate fi șters");

        _db.DeleteUser(user.Id);
    }

    /// <summary>
    /// Retrieves all users from the database.
    /// </summary>
    public List<User> GetUsers()
    {
        return _db.SelectAllUsers();
    }
}
```

3. Singleton:



FormAddProduct:

```
private static FormUpdatePass _instance = null;

...

private FormUpdatePass()
{
    InitializeComponent();
    ConnectButton(buttonUpdate);
}

public static FormUpdatePass GetInstance()
{
    if (_instance == null || _instance.IsDisposed)
    {
        _instance = new FormUpdatePass();
    }
    else
    {
        _instance.BringToFront();
```

```
    }
    return _instance;
}
 ...
```

USER GUIDE

This **help guides** you through the core features of the **Auto Parts Store Management application**: a Windows desktop solution designed to streamline your shop's operations by digitizing inventory control, sales processing, and user account management.

What the App Does

- **Inventory Management**: Add new parts, restock existing items, and update pricing.
- **Sales Processing**: Sell parts quickly and accurately, with real-time stock updates.
- **User & Role Management**: Secure, role-based access for Administrators, Stock Managers, and Sales Clerks.

Who Should Read This

This help is organized into three chapters, each tailored to your role and permissions:

- Admin's Guide: Full control over user accounts.
- Stockman's Guide: Manage product catalog, stock levels, and pricing.
- Seller's Guide: Process sales and view current inventory.

All three guides begin with the same essential step: **logging into the system** (fig 1). Before an Administrator, Stock Manager, or Sales Clerk can access their respective dashboards—whether to manage user accounts, update inventory, or process sales—all users must first authenticate themselves. Below is an overview of how to log in.

LOGGING IN:

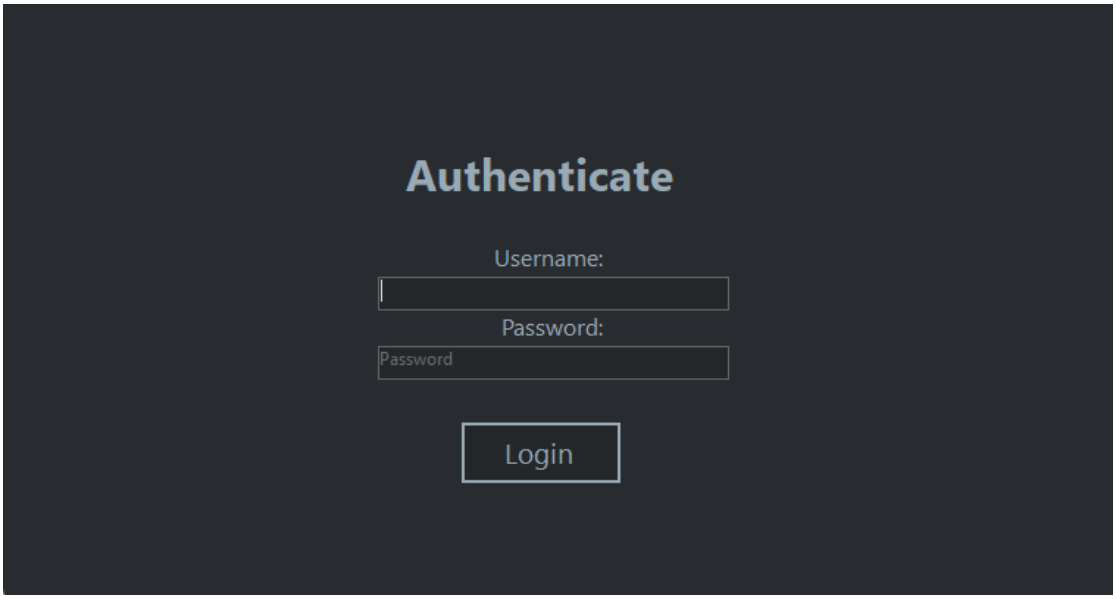


Fig 1. Login screen

Firstly, you must type in your username into the first (1) textbox and the password into the second (2) textbox. After you have typed them, you press the Login (3) button -> Fig 1.1

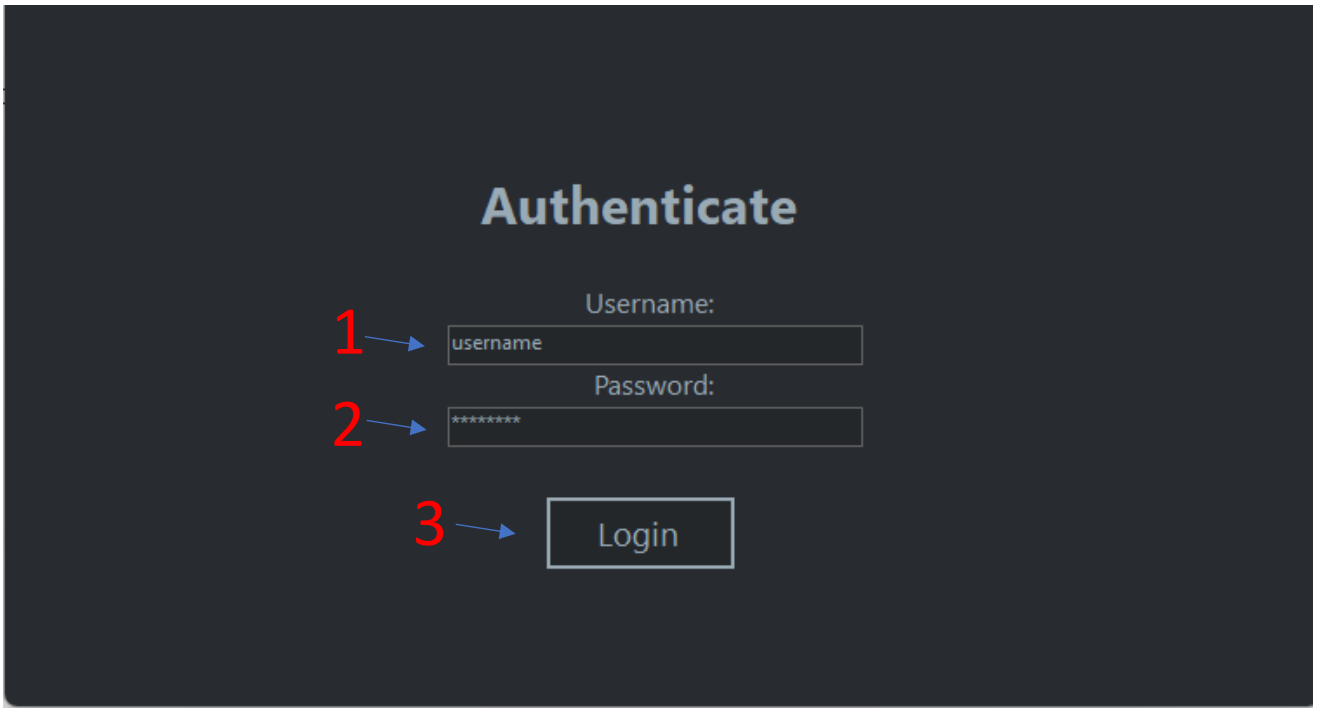


Fig 1.1 Log in tutorial

If the password or the usernames were incorrectly typed, you will get the following pop-up (Fig 1.2). Do not panic! Press the OK (1) button and try again. If you continue on getting errors, contact the administrator of the service.

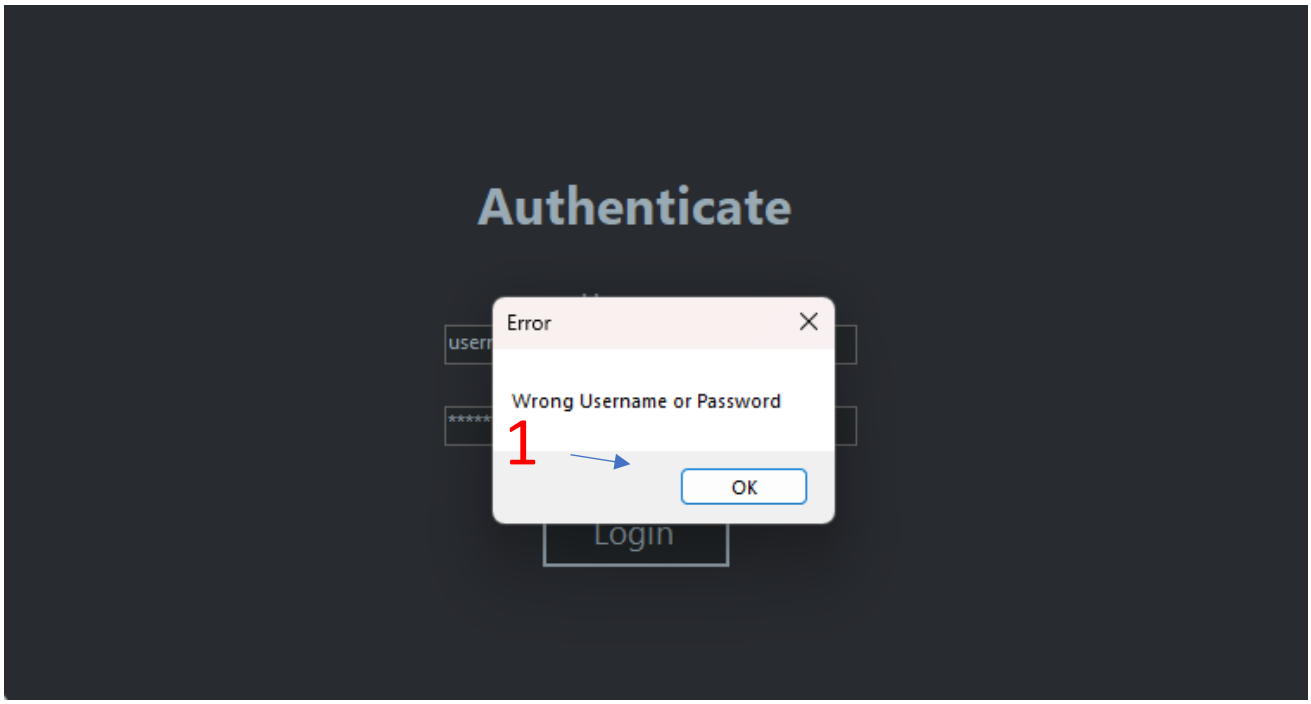


Fig 1.2 Log in failed screen

1. Administrator Guide

Welcome to the **guide dedicated to the Administrator role!** Here you'll find everything you need to manage the users of the application.

Prerequisites

- You must be logged in with an **Administrator account.**

Contents

- Adding a new user
- Viewing & Refreshing the User List
- Deleting a User
- Updating an user's password

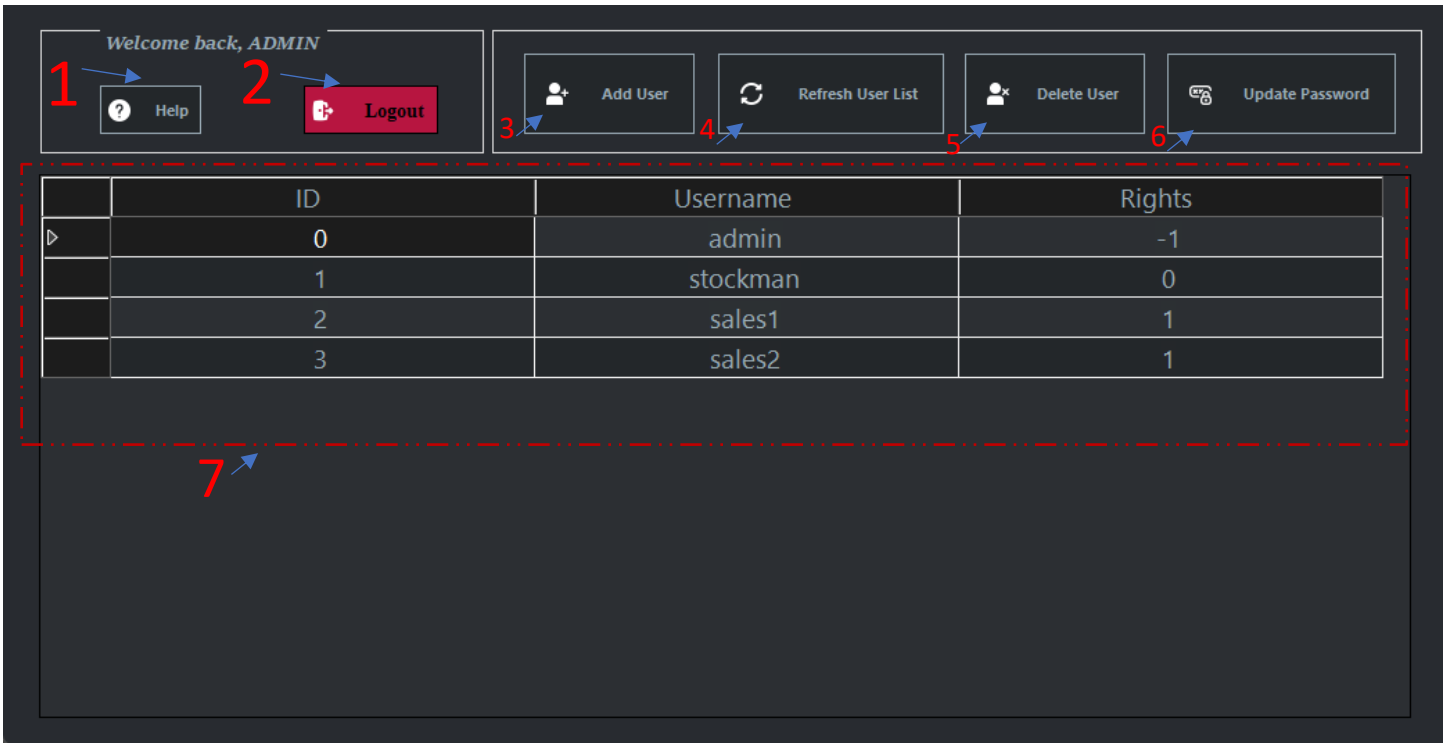


Fig 2. Administrator dashboard

Figure 2 explained:

1. Help button: Opens the help menu
2. Logout button: You log out from your account
3. Opens the add an user button
4. Refreshes the user list (7)
5. Delets an user
6. Updates and user's password
7. Shows the user list

1.1 Adding a new user

Press the Button number 3 from Fig 2. Afterwards, you will see the following pop-up (fig 2.1):

Fig 2.1

In order to add a new user, complete the form as follows:
a) **Username** (1) – a unique identifier (required)

b) **Password** (2) - temporary password for the new account (required)

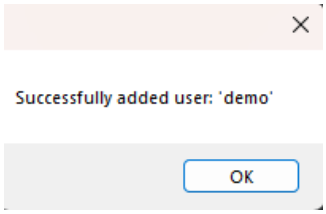
c) **Role Code** (3), as follows:

i) 0 for a **Stock Manager**

ii) 1 for a **Sales Clerk**

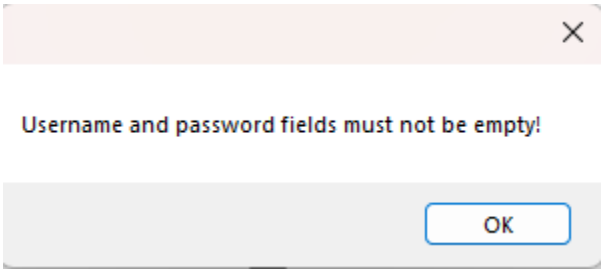
d) Click the **Add** (4) button.

e) If the input is valid, you'll see a confirmation:



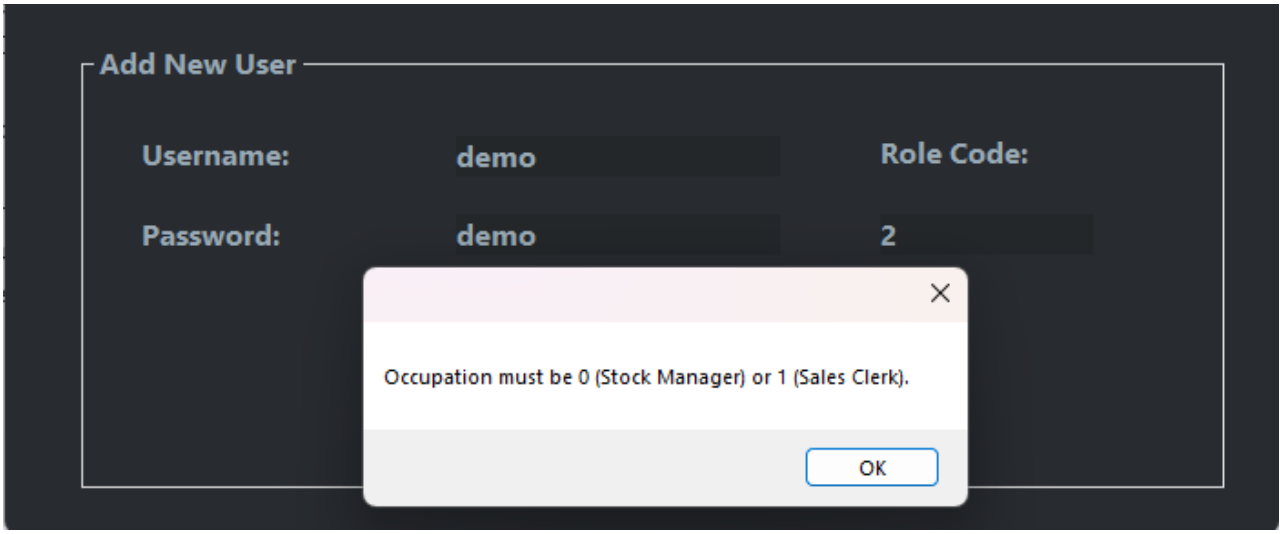
f) Back on the Menu, click **Refresh User List** (Fig 2 button 6) to confirm the user has been added.

Possible errors:
1. Empty Username or Password



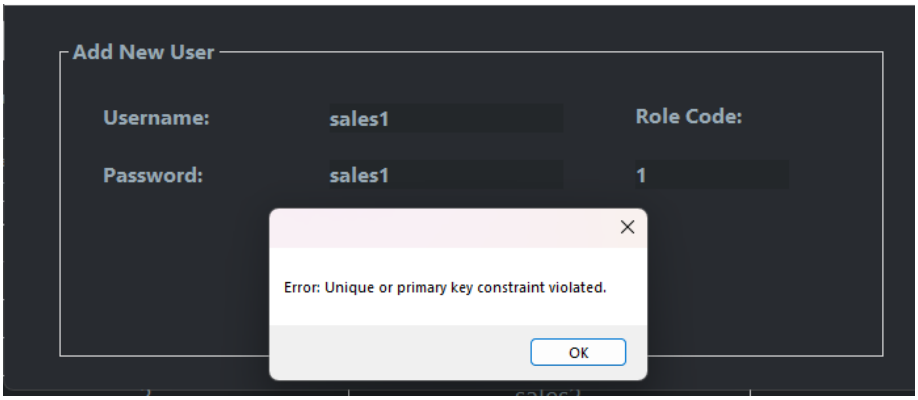
When you see this, you know you must type in the username and the password for the process of registering a user to be completed.

2. Invalid Role Code



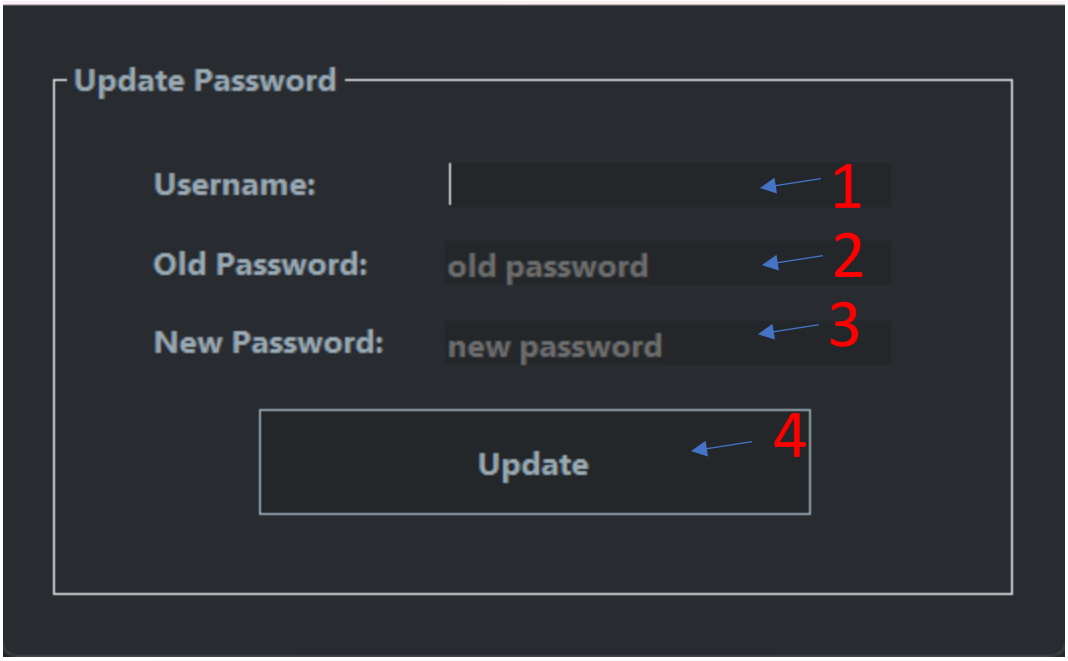
When you see this pop-up, you know you have to add valid Role Codes.

2. Same username as an existing account: The username must be unique

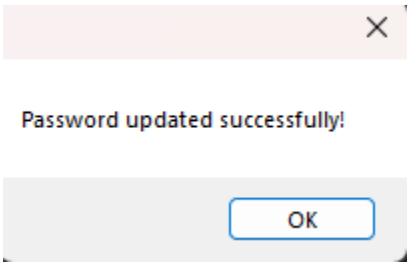


1.2 Updating an user's password

1. From the Menu toolbar, click the **Update Password button** (6 Fig 2), then the following popup will appear:



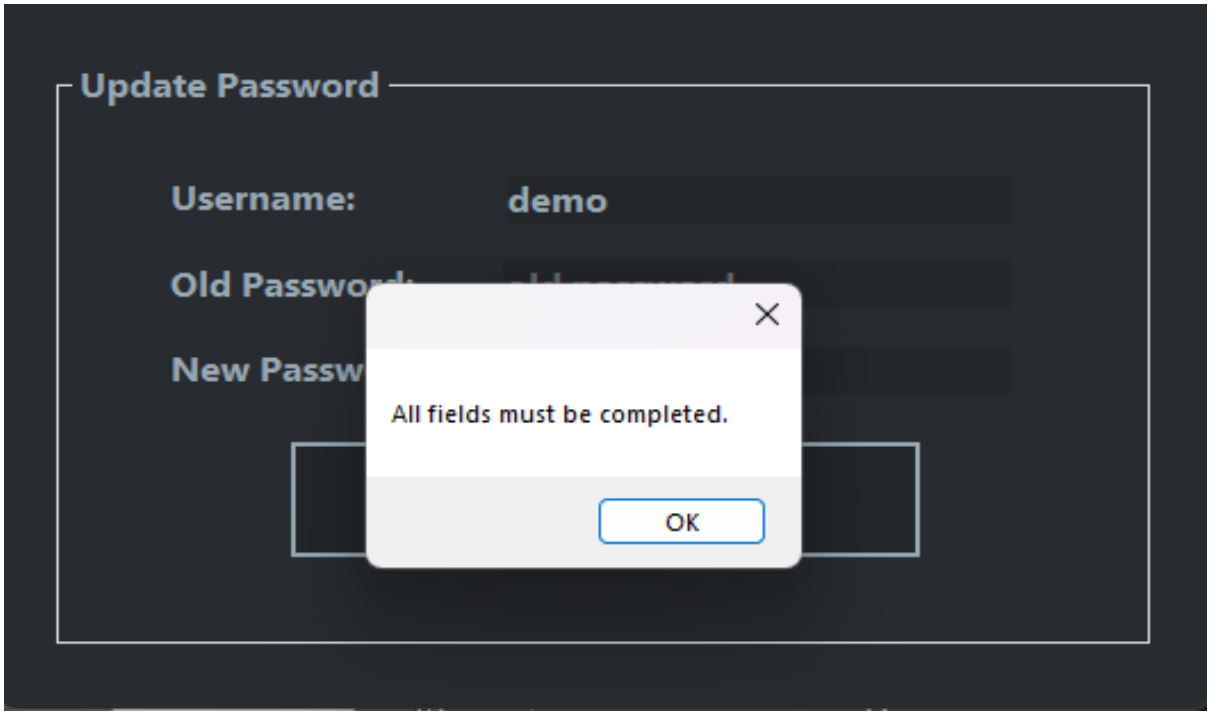
2. In the Update Password dialog, complete all fields:
    1. Username (1) – the user's unique identifier (required)
    2. Old Password (2) – the user's current password (required)
    3. New Password (3) – the password you want to assign (required)
3. Click UPDATE (4).
4. If the input is valid, a confirmation message appears:



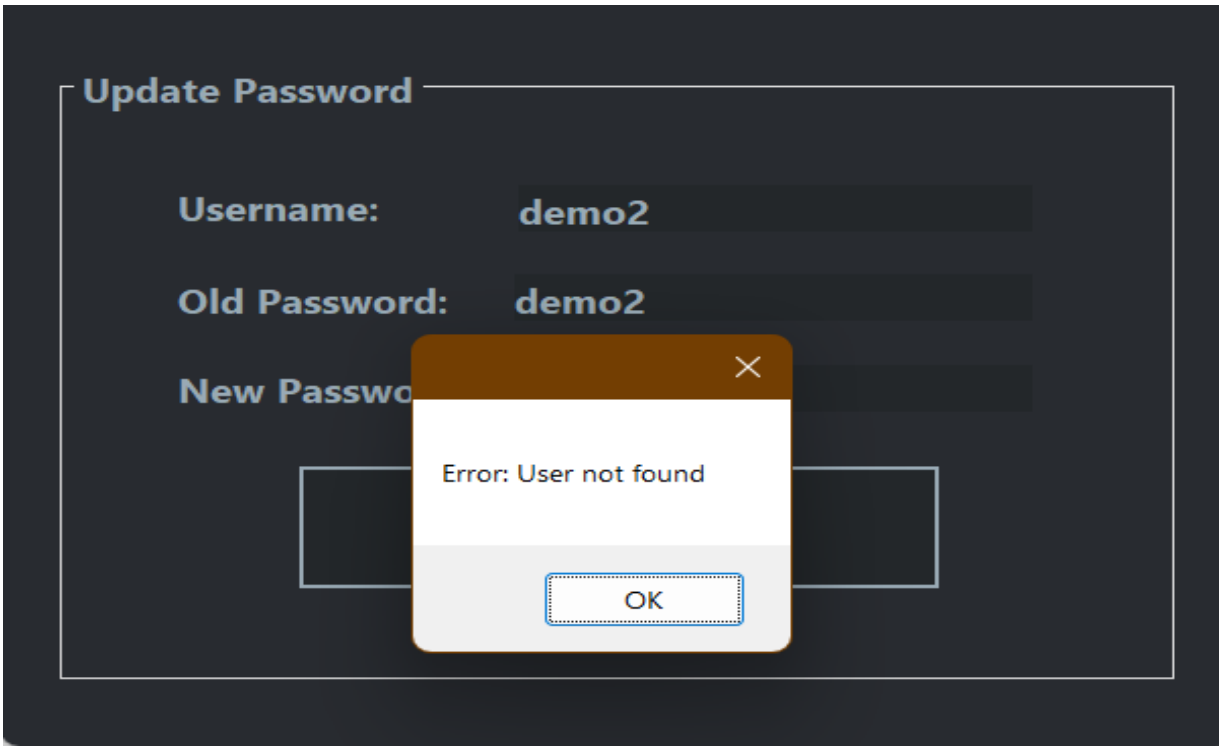
5. The dialog closes automatically.

Validation & Error Messages

- Any Field Left Blank



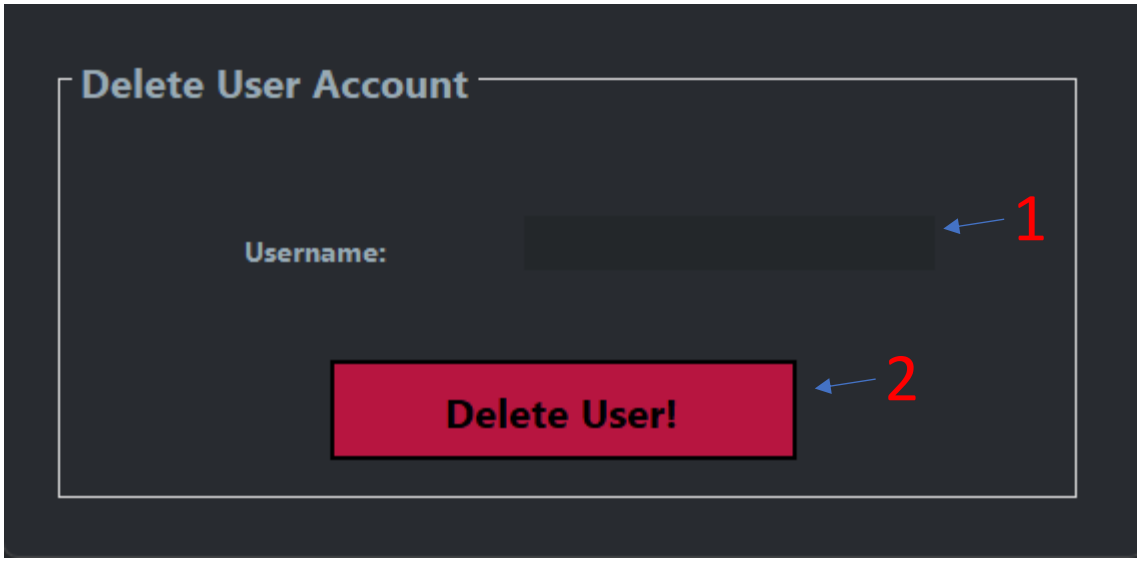
- Invalid username (non-existent)



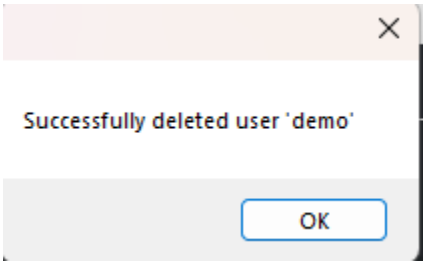
After each of those errors, Consider the pop-up message and fix the problem.

1.3 Deleting an user

1. From the Menu toolbar, click the **Delete User button (Fig 2 button 5),** then the following pop-up will appear:



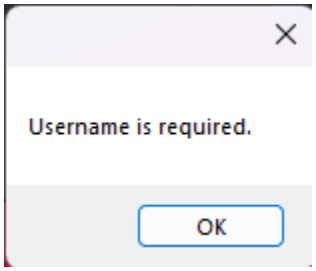
2. In the Delete User dialog:
    1. Enter the **Username** (1) of the account you wish to delete (required).
3. Click **Delete** (2).
4. If the input is valid, a confirmation message appears:



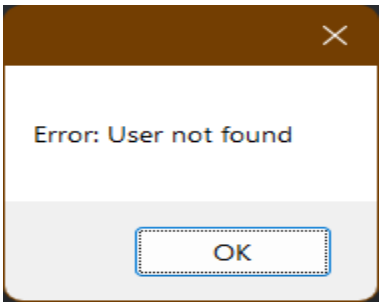
5. The dialog closes automatically.
6. Back on the Menu, click Refresh User List to confirm the user has been removed.

Validation & Error Messages

- Empty Username Field



- User not found



2. Stockman's Guide

Welcome to the Stockman's Guide! Here you'll find all you need to manage inventory:

- adding new parts, restocking existing items, updating prices, selling and keeping your product list up to date.

Prerequisites

- You must be logged in with a **Stockman account.**

Stockman's menu:

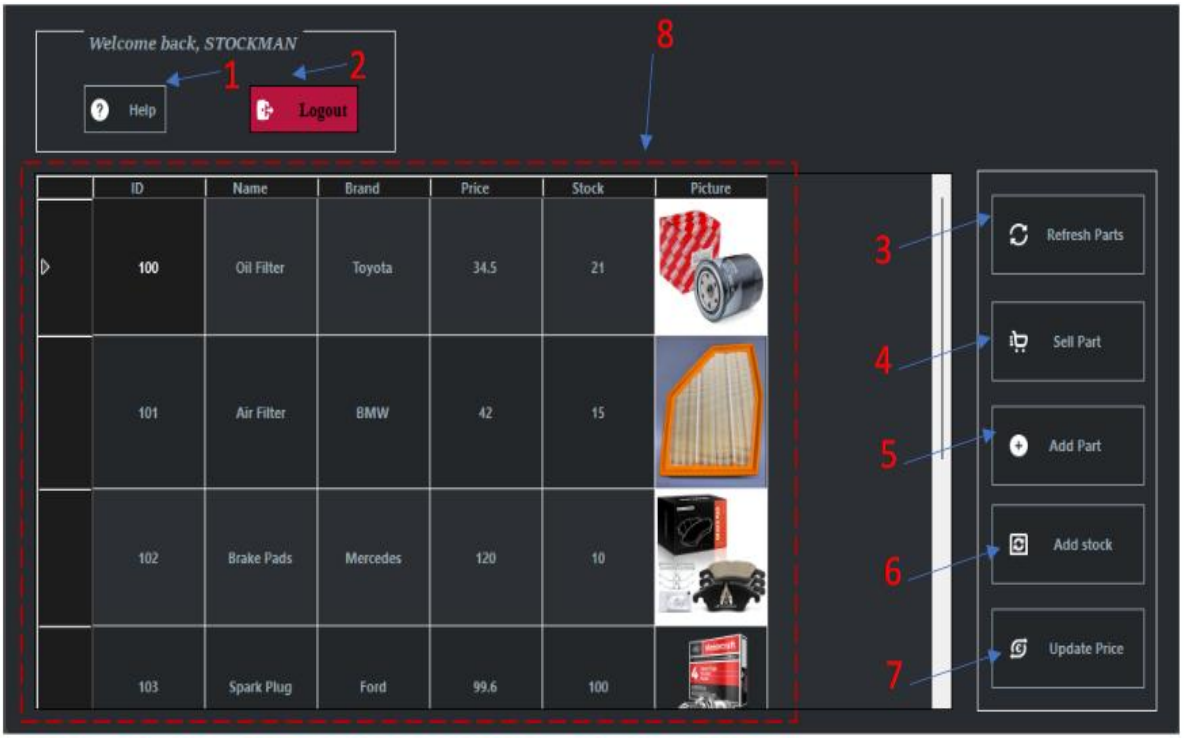


Fig.3.

Explaining of the buttons:

1. Opens the help menu
2. Logout button
3. Refreshes the part list
4. Sell a part
5. Add a part
6. Add to the stock of a part

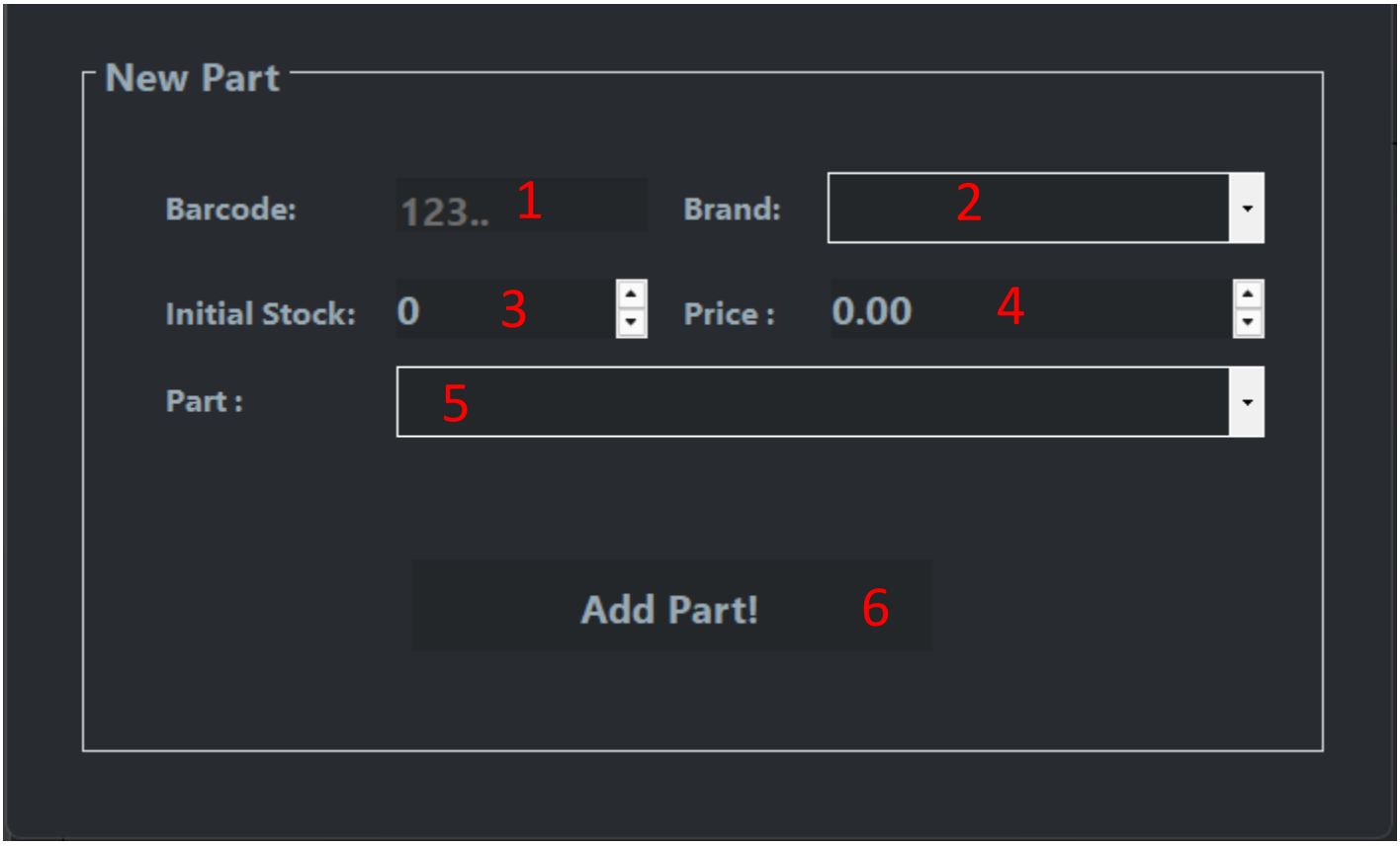
7. Update the price of a part

2.1. Adding a new Part

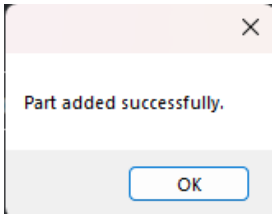
Allows a Stock Manager to create a new auto part record in the system, specifying its brand, model, barcode, price, and initial stock.

Steps to Follow

1. On the Menu toolbar, click **Add New Part** (**Fig 3. Button 5**)



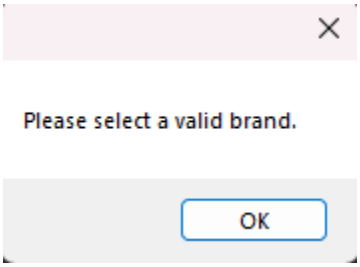
2. In the New Part dialog, complete all fields:
    1. **Brand** (2) – select a valid manufacturer from the dropdown (required).
    2. **Part** (5) – once a brand is selected, choose a model from the Part dropdown (required).
    3. **Barcode** (1) – enter the part's numeric code (required).
    4. **Price** (4) – set the unit price (must be > 0).
    5. **Initial Stock** (3) – set the starting quantity (must be ≥ 0).
3. Click **Add** (6).
4. If the input is valid, a confirmation message appears:



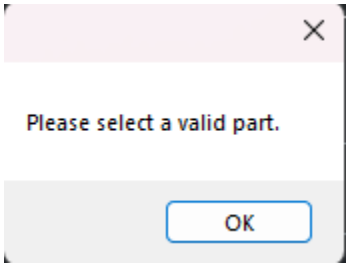
5. The dialog closes automatically.
6. Back on the Menu, click **Refresh part List (Fig3 button 3)** to view your new part.

Validation & Error Messages

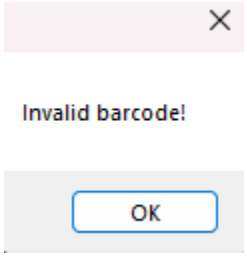
- Invalid Brand Selection



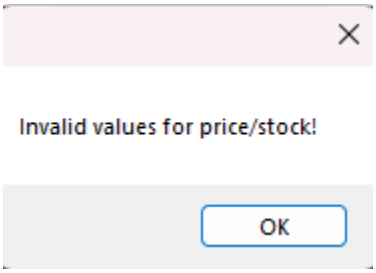
- Invalid Part Selection



- Barcode Not a Number



- Price ≤ 0 or Stock < 0



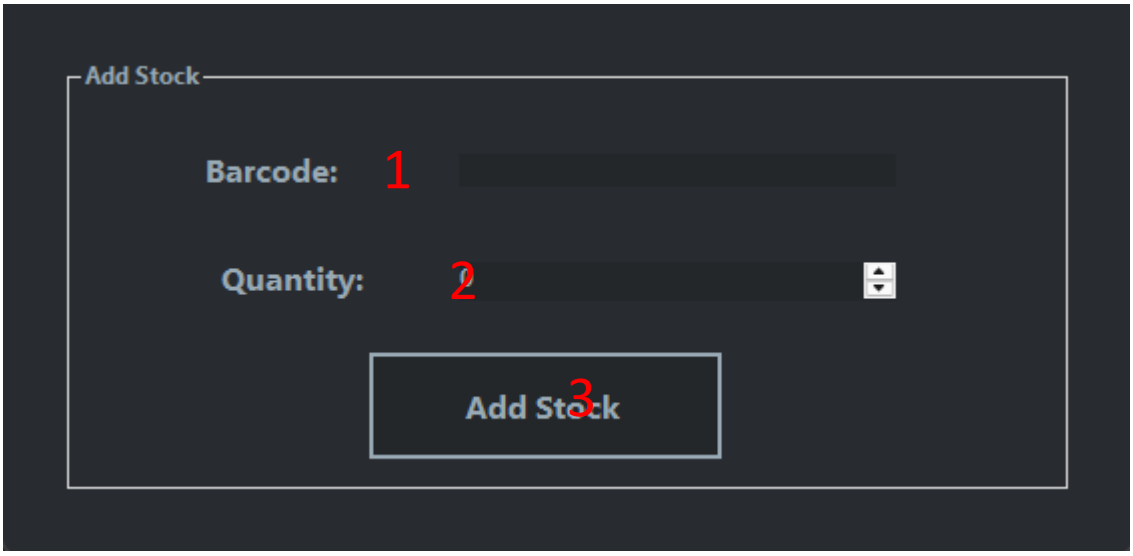
Follow the message from the errors to solve them.

2.2. Adding to the stock

Allows an Stock Manager to increase the inventory count for an existing auto part by specifying its barcode and the quantity received.

Steps to Follow

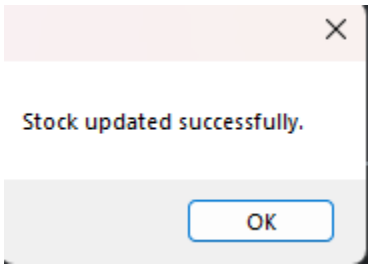
1.On the **Menu** toolbar, click **Add to Stock (5),** then the following pop-up will appear:



2.In the Add Stock form, complete:

1. **Barcode(1)** – enter the part's numeric code (required).
2. **Quantity(2)** – enter the number of units to add (must be greater than 0).
3. Click **Add to Stock(3)**.

4.If the input is valid, a confirmation message appears:

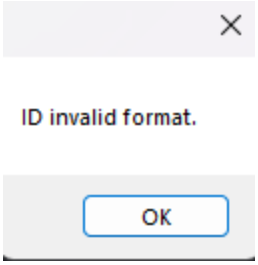


5.The dialog closes automatically.

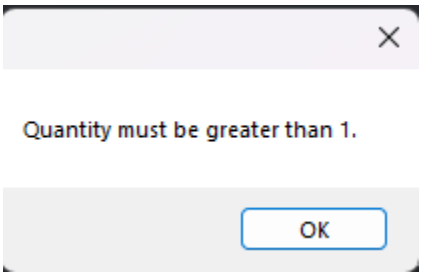
6.Back on the Menu, click **Refresh Product List (Fig3 button 3)** to view your new part.

Validation & Error Messages

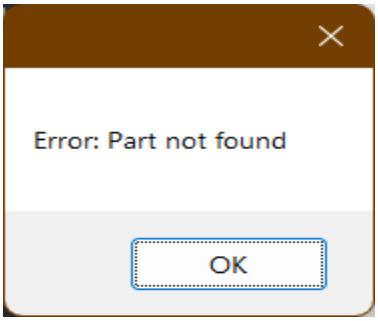
- Invalid Barcode



- Invalid Quantity



- Unknown product:



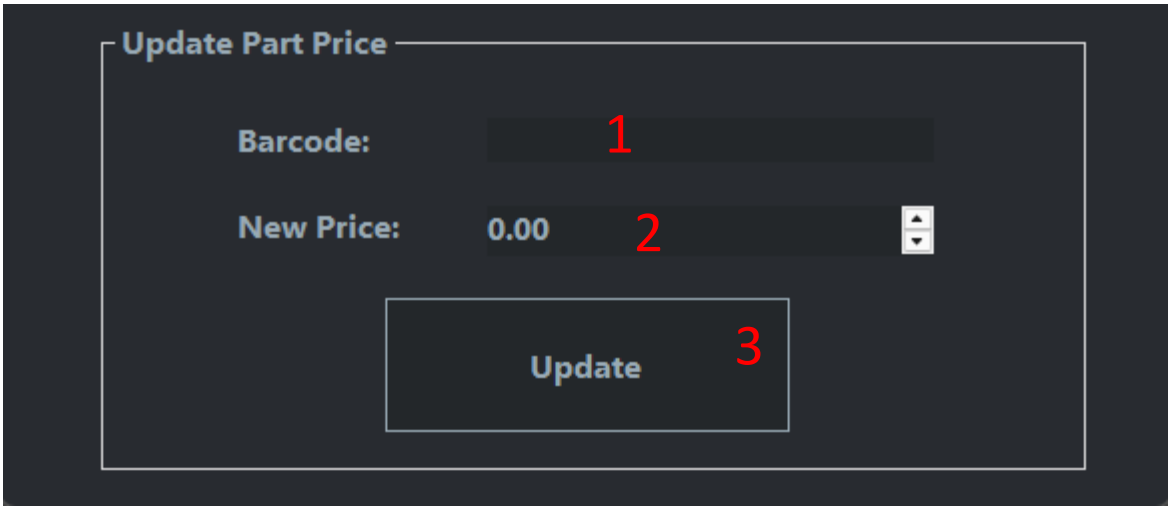
In order to fix the errors, you have to read them and try to resolve the problem displayed in it.

2.3. Updating a part's price

Allows a Stock Manager to change the unit price of an existing auto part.

Steps to Follow

1.On the Menu toolbar, click **Update Price** (Fig 3 button 3), then the following pop-up will appear:

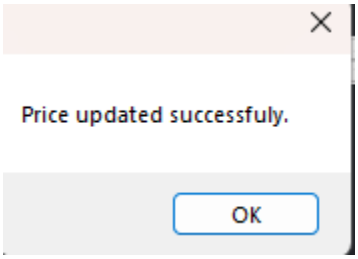


2.In the **Update Part Price** dialog, complete:

1. **Barcode (1)** – enter the part's numeric ID (required).
2. **New Price (2)** – enter the updated unit price (must be > 0).

3.Click **Update (3)**.

4.If the input is valid, a confirmation message appears:

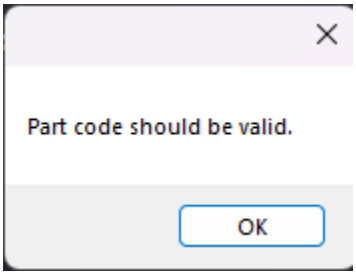


5.The dialog closes automatically.

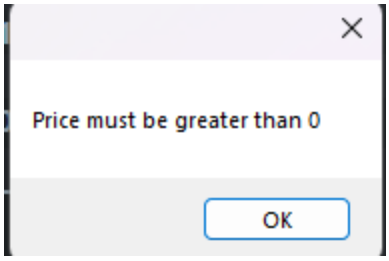
6.Back on the Menu, click **Refresh Parts** (Fig 3 Button 3) to see the new price in the product list.

Validation & Error Messages

- Invalid Barcode Format



- New Price ≤ 0



In order to fix the errors, you have to read them and try to resolve the problem displayed in it.

2.1.1 Sales Clerk + Stockman: Selling a part

As a stockman, press the Fig 3 button 4 in order to initiate a transaction.

As a sales Clerk, in order to initiate a transaction, you will need to:

a) Log in, as displayed in the beginning
b) Then, you will see the following panel:

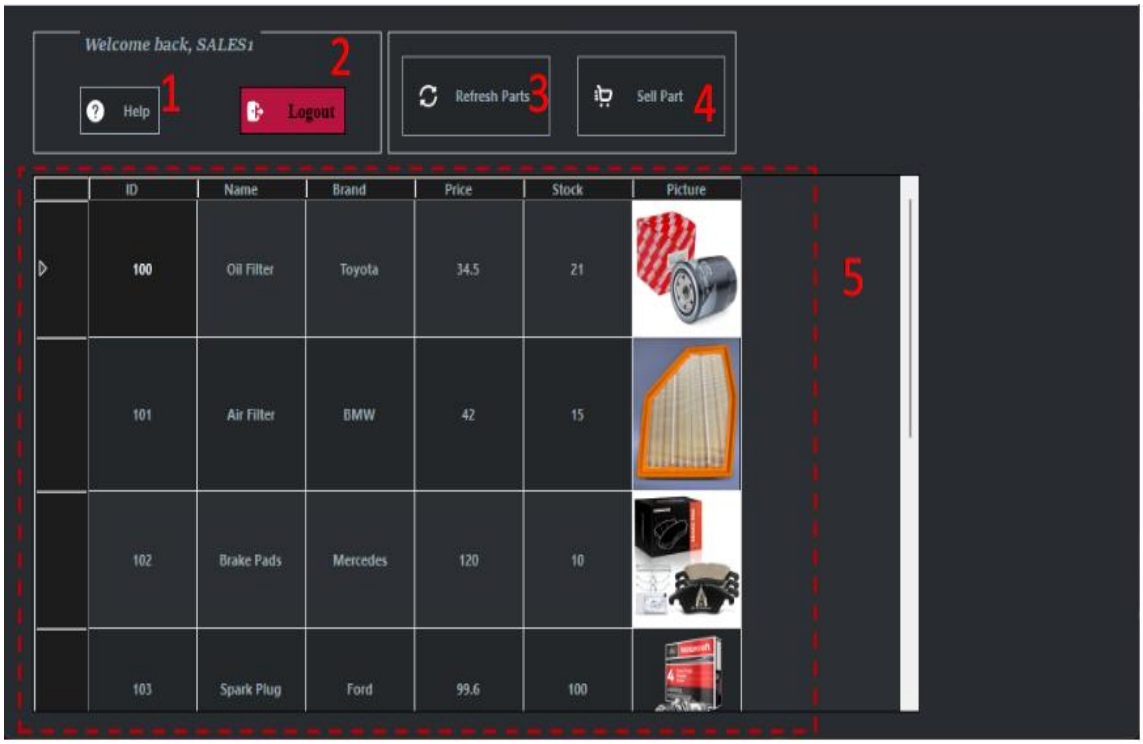


Figure 4.

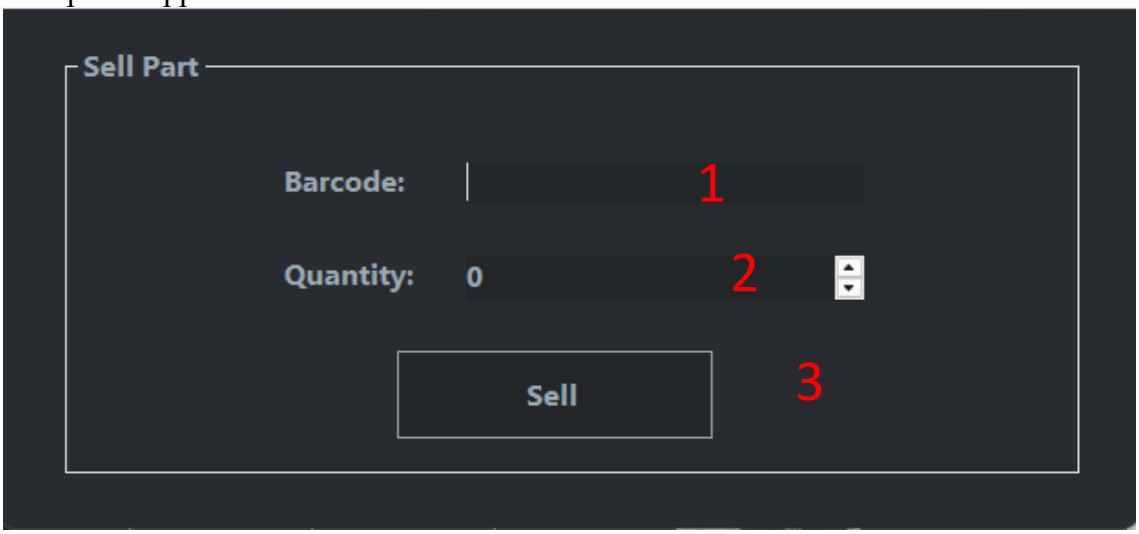
1. Help button: Opens the help button
2. Logout: Logout from the current account
3. Refresh Parts: refreshes the part list, after a sale
4. Sell Part: Sells a part
5. Part List

3.1. Selling a part

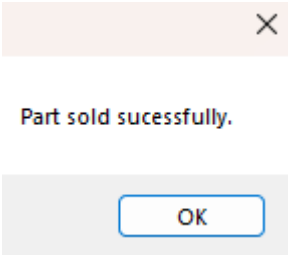
As a Stockman, press the button 4 from figure 3. As a Sales Clerk, press the button 4 from figure 4.

Steps to Follow

1. On the **Menu** toolbar, click Sell Part, as explained in the section from above, then the following pop-up will appear:



2. In the **Sell Part** dialog, complete all fields:
    1. **Barcode(1)** –enter the part's numeric code (required).
    2. **Quantity(2)** - enter the number of units to sell (must be ≥ 1).
3. Click **Sell(3)**.
4. If the input is valid, a confirmation message appears:

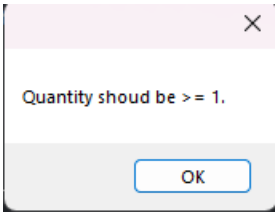


5. The dialog closes automatically.
6. Back on the **Menu**, click **Refresh Product List**(**Fig 3 button 3 for Stockman**, **Fig 4 button 3 for Sales Clerk**) to view your new part.

Validation & Error Messages

• Invalid Barcode

• Quantity < 1



In order to fix the errors, you have to read them and try to resolve the problem displayed in it.